Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

DATA PLATFORM REGIONALE
DOCUMENTO DI ARCHITETTURA

Autore:	Riccardo Di Dio, Stefano Parodi	15/11/2023
Rivisto:	Mauro Giaccardi, Andrea Pasotti	27/12/2023
Approvato:	Andrea Pasotti	27/12/2023
Versione:	1	
Distribuito:	Regione Liguria	

Contents

1 - Introduzione 5
2 - Requisiti 6
2.1 - Requisiti Utente 6
2.2 - Requisiti Tecnici 10
3 - Sintesi dell'architettura 19
4 - Descrizione dell'architettura 22
4.1 - Descrizione dell'architettura 22
4.2 - Componente di Object Storage e Data Virtualization 25
4.3 - Componente di Orchestration 28
4.4 - Componente di Data Analysis 33
4.5 – Componenti Catalogo Regione Liguria e Catalogo Open Data 36
4.6 - Componenti di fruizione: Integrazioni Cartografiche, Portale CKAN, Open Data Downloader e API end-point 36
4.7 – Componenti di Logging e Monitoraggio 38
4.8 – Diagramma generale 38
5 - Descrizione dei componenti 40
5.1 - Componente MinIO 40
Object storage 40
MinIO 41
Funzionalità principali 41
Bucket Notifications 42
Architettura 43
5.3 – Componenti Iceberg e Project Nessie 45
Table format 45
Apache Iceberg 45
Table Repository 46
Project Nessie 46
5.4 – Componente Dremio 49
Layer di Data Access 49
Dremio 49

Caratteristiche principali 49
5.5 Componente Orchestrator 52
Mage 52
Concetti chiave 52
Apache Airflow 56
Concetti chiave 56
Nifi 60
Concetti chiave 60
Architettura Nifi 61
Proposta di dispiegamento 62
5.6 – Componente Kafka 64
Apache Kafka 64
Architettura 65
5.7 – Componente dbt 67
DBT (Data Build Tool) 67
DBT all'interno della nostra infrastruttura 68
5.8 – Componente Spark 69
Apache Spark 69
Ruolo e Funzionalità: 69
Caratteristiche Tecnologiche: 69
Struttura: 69
Architettura di Spark: 70
5.9 – Componente Jupyter e Zeppelin 72
5.10 – Componente Data Catalog 76
Data Catalog 76
DataHub 76
Architettura 77
Open Metadata 80
Architettura 80
5.11 – Componente Opensearch 83

Monitoraggio 83
Opensearch 83
Architettura 84
5.12 – Componenti Cartografiche 85
GDAL 85
DuckDB 85
Geopandas 86
Leafmap 87
deck.gl 87
PBI-Carto 88
5.13 – Componente Superset 89
Apache Superset 89
5.14 – Componente Opendata Downloader E Api end-point 91
Componente downloader 91
Componente Api REST 93

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

1 - Introduzione

Il presente documento ha lo scopo di descrivere nel dettaglio tecnico l'architettura della nuova data platform di Regione Liguria. Questa nuova infrastruttura ha l'obiettivo principale di migliorare il controllo e la gestione del patrimonio informativo a servizio del cittadino e delle pubbliche amministrazioni.

Questo renderà possibile una fruizione del dato più consapevole e di più ampiamente disseminabile, permettendo un maggior utilizzo e visibilità dei dati conservati nel corso degli anni.

In quest'ottica sarà fondamentale permettere agli utenti di trovare le informazioni ricercate con maggiore accuratezza, supportate da una metadatazione che possa abilitare un modello virtuoso di interazione e conservazione del dato.

A corredo degli strumenti tecnici saranno definiti dei processi organizzativi su molteplici livelli, che permetteranno di definire e applicare standard di corretta gestione del ciclo di vita del dato in ogni sua fase.

La lettura di questo documento, si articola in capitoli in cui il livello di dettaglio aumenta progressivamente.

Nei Requisiti (Capitolo 2) viene elencato l'insieme di input delle caratteristiche che il sistema deve avere e i vincoli che deve rispettare.

Nella Sintesi dell'architettura (Capitolo 3) viene presentata una visione di alto livello del sistema, per comprendere le potenzialità generali del disegno.

Nella Descrizione dell'architettura (Capitolo 4) vengono descritte le interazioni tra le macro-componenti logiche che permettono l'implementazione delle funzionalità del sistema.

Nella Descrizione dei componenti (Capitolo 5) sono dettagliate le caratteristiche tecnologiche e i ruoli all'interno dell'architettura di tutte le componenti individuate.

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

2 - Requisiti

I requisiti che hanno guidato l'analisi e la stesura di questo documento, sono stati divisi in due macrocategorie che rappresentano gli obiettivi e i punti di vista dei diversi stakeholder. I requisiti utente descrivono le caratteristiche che il sistema deve avere dal lato degli utenti finali e dello stakeholder Regione Liguria. I requisiti tecnici definiscono le necessità tecnologiche necessarie alla costruzione della data platform.

Di seguito si trovano le liste che contengono i riferimenti univoci al requisito.

2.1 - Requisiti Utente

Nei requisiti utente abbiamo estratto le necessità degli stakeholder di presentazione e utilizzo del sistema e le abbiamo messe a sistema con le esperienze maturate nel corso degli anni. Ogni requisito utente è considerato necessario e deve quindi essere rispettato in fase di definizione e implementazione dell'architettura.

RU01

La data platform deve essere abilitante alla produzione di servizi orientati alla gestione e alla fruizione del dato.

RU01.1

I carichi di lavoro di competenza della data platform non devono essere di tipo gestionale. Le funzionalità offerte sono orientate a sostenere carichi di tipo analitico e di reportistica.

RU02

Le interfacce verso gli utenti devono essere conformi alle direttive di accessibilità nazionali e aziendali.

RU03

Le risposte dovranno essere fornite in tempi rapidi compatibilmente con il tipo di elaborazione richiesta e comunque nell'ordine dei secondi. Eventuali elaborazioni più complesse che

richiedono tempi più lunghi dovranno essere segnalate all'utente e, in caso di esecuzione in modalità batch, il sistema dovrà inviare un avviso all'utente ad elaborazione terminata.

RU04

La data platform deve essere in grado di gestire il dato in maniera conforme alle direttive di legge espresse dal GDPR.

RU05

La data platform deve essere conforme alle linee guida AGID sulla pubblicazione e condivisione dei dati aperti.

RU06

Le interfacce verso gli utenti devono poter essere protette e autenticate attraverso le modalità standard nazionali dove necessario.

RU07

Le interfacce di interoperabilità tra i sistemi che la data platform espone verso l'esterno devono poter essere veicolate attraverso il sistema PDND.

RU08

La data platform deve facilitare le integrazioni tra dati cartografici e dati alfanumerici.

RU09

La data platform deve offrire interfacce per la visualizzazione e l'analisi dato interattiva.

RU10

La data platform deve essere responsabile della qualità e consistenza dei dati al suo interno attraverso consolidati processi di data governance.

RU11

La data platform deve avere almeno la medesima copertura funzionale e di SLA delle piattaforme che andrà a sostituire.

RU12

La data platform deve poter ospitare analisi selezionate provenienti dal datawarehouse regionale in dismissione.

RU13

La data platform deve poter ospitare le analisi selezionate che verranno migrate da architetture analitiche in obsolescenza.

RU14

La data platform deve essere in grado di gestire l'alimentazione e il mantenimento dei sistemi di interoperabilità nazionali ed europei secondo gli standard in vigore.

RU15

La scelta delle componenti deve privilegiare soluzioni open-source o comunque con il più basso impatto sui costi ricorrenti.

RU16

La data platform dovrà offrire strumenti per la metadatazione e catalogazione semantica dei dataset.

RU17

La data platform deve offrire la possibilità di catalogare metadati:

- per gruppi di contesto di dataset
- per singolo dataset
- per proprieta' e valori notevoli del dataset

RU18

La data platform deve fornire servizi per scaricare i dati disponibili indipendentemente dalla mole, in modalità sincrona o asincrona a seconda dei livelli di servizio attesi/richiesti.

RU19

Le interfacce sui dati aperti verso gli utenti e i sistemi devono essere resi disponibili tramite Api REST.

RU20

La data platform deve permettere l'estensione delle funzionalità di integrazione tra reportistica e applicazioni.

RU21

L'autenticazione e l'autorizzazione deve avvenire attraverso i sistemi di identity management e le politiche attualmente in uso.

RU22

La data platform deve facilitare la definizione e il mantenimento di politiche per la gestione delle edizioni, versioni, e correzioni dei dataset sia per i dati interni che per quelli esposti all'esterno.

RU23

Le funzionalità core della data platform dovranno essere disponibili entro il primo anno dall'inizio del progetto.

RU24

La data platform deve guidare l'ammodernamento dei processi di gestione del dato mettendo a disposizione il riuso delle scelte tecnologiche e di governance.

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

2.2 - Requisiti Tecnici

I requisiti tecnici sono l'espressione delle necessità tecnologiche individuate durante l'analisi, messe a sistema con l'esperienza maturata negli anni e l'uso/sviluppo di sistemi similari.

Per quanto riguarda la notazione, tutti i requisiti che presentano un asterisco (*) alla fine del nome di riferimento, sono da considerarsi imprescindibili. Gli altri sono da considerarsi estremamente importanti, ma con priorità inferiore e che potrebbero essere raggiunti in altre modalità.

*RT01**

La data platform deve offrire capacità di approvvigionamento dei dati a partire dai sistemi informativi gestionali e analitici attualmente in uso.

*RT02**

La data platform deve offrire capacità di pulizia, preparazione e armonizzazione dei dataset.

*RT03**

La data platform deve offrire strumenti che abilitino all'analisi e alla trasformazione dei dataset in maniera più efficiente ed efficace rispetto ai sistemi gestionali e analitici attualmente in uso.

*RT04**

La data platform deve prevedere funzionalità di orchestrazione, automazione e schedulazione dei processi per gestire tutte le fasi del ciclo di vita del dato.

*RT05**

La data platform deve esporre schemi e modelli che descrivano la struttura e le relazioni tra diversi dataset in un repository centrale.

*RT06**

Lo strato di persistenza della data platform deve offrire almeno un protocollo consistente verso tutte le componenti dell'architettura.

*RT07**

Lo strato di persistenza della data platform deve essere distribuito su nodi decentralizzati ed accessibile attraverso un unico protocollo.

*RT08**

La data platform deve offrire una soluzione di persistenza in grado di gestire i dati provenienti da fonti eterogenee e di diverso formato.

*RT09**

La data platform deve fornire un'interfaccia per l'accesso e l'interrogazione dei dati indipendentemente dalla provenienza e dal formato.

*RT10**

La data platform deve prevedere strumenti di data governance e di data observability su tutto il ciclo di vita del dato.

*RT11**

La data platform deve essere in grado di rendere il dato facilmente reperibile attraverso strumenti di ricerca e robusta metadatazione.

*RT12**

La data platform deve fungere da punto di raccolta e distribuzione centrale per i dati di interesse trasversale, curarne il mantenimento e la qualità al fine di riutilizzarli in diversi contesti.

*RT13**

Tutte le componenti della data platform devono essere in grado di fornire i loro log ad un sistema centrale che deve fungere da punto di raccolta degli eventi del sistema sia a livello applicativo che infrastrutturale.

*RT14**

La data platform deve fornire una base di integrazione per le attuali soluzioni di reportistica visuale, ed estendere queste capacità con strumenti aggiuntivi.

*RT15**

La data platform deve mettere a disposizione delle interfacce o dei servizi che permettano ad altri sistemi di usufruire delle funzionalità analitiche offerte dall'architettura.

*RT16**

La data platform deve offrire strumenti per la realizzazione di servizi orientanti alla fruizione del dato e che facilitino l'integrazione con il sistema open data di regione.

*RT17**

La data platform deve essere in grado di gestire l'intero ciclo di vita del dato mediante fasi definite in processi di pipeline.

*RT18**

La data platform deve rendere disponibili meccanismi per l'alta affidabilità delle pipeline quali la ridondanza attraverso repliche o la distribuzione attraverso clustering.

*RT19**

La componente di orchestrazione della data platform deve permettere la definizione di gerarchie e dipendenze (temporali e di contesto) tra le fasi delle pipeline.

*RT20**

La data platform deve offrire funzionalità di monitoraggio e controllo delle pipeline e dei processi, quali:

- capacità di auditing
- visualizzazione di metriche
- gestione di alerts
- propagazione di notifiche.

*RT21**

La data platform deve offrire gli strumenti necessari alla gestione di fallimenti e comportamenti anomali nelle pipeline.

*RT22**

La data platform deve avere strumenti e processi per la metadatazione tali da offrire interfacce accessibili in quante più fasi possibili, soprattutto nelle fasi di acquisizione e di trasformazione.

*RT23**

Il processo di metadatazione deve essere semplice e mantenibile nel tempo, il più possibile automatizzabile secondo regole ben definite, sia generali che specifiche per il dominio e per il dataset.

*RT24**

La data platform deve permettere ove necessario l'impiego di protocolli di encryption per il trasferimento dei dati al suo interno.

*RT25**

La data platform deve permettere ove necessario di definire politiche di accesso ai dataset granulari a livello di riga o di colonna.

*RT26**

La data platform deve supportare nativamente capacità di versionamento dei dataset e degli schemi al suo interno.

*RT27**

La data platform deve permettere di automatizzare e mettere in sequenza task e job di analisi, cleaning, processing e modellazione dei dati.

*RT28**

La data platform deve esporre un'interfaccia che copra le funzionalità di interrogazione compliant con gli standard ANSI SQL indipendentemente dal supporto e dal formato di memorizzazione del dataset.

*RT29**

La data platform deve poter essere interrogata in maniera uniforme sia dalle componenti alla base che dai consumatori.

*RT30**

La data platform deve poter ospitare dataset, formati e analisi di tipo geospaziale permettendo:

- trasformazione dei dati in diversi formati
- analisi squisitamente sulla dimensione geospaziale
- analisi geospaziali integrate con altre dimensioni

*RT30.1**

La data platform deve permettere l'integrazione di funzionalità cartografiche negli strumenti di reportistica attualmente in uso e la possibilità di sviluppare client cartografici specifici maggiormente orientati alla geospatial analytics.

*RT31**

La data platform deve offrire almeno un motore agnostico rispetto ai formati di memorizzazione e tabellari e agli standard analitici di interrogazione.

*RT32**

La data platform deve almeno garantire risposte in fase di analisi che siano compatibili con il calcolo degli indicatori giornalieri nel caso d'uso della qualità dell'aria.

*RT33**

La data platform deve essere abilitante alla creazione di modelli analitici di machine learning, apprendimento e inferenza.

*RT34**

La data platform deve comunicare prevalentemente attraverso Api REST offrendo anche interfacce verso l'esterno che seguano le medesime caratteristiche.

*RT35**

La data platform deve permettere l'autenticazione delle componenti di sviluppo e gestione almeno attraverso directory LDAP.

*RT36**

Le componenti della data platform devono prevedere la divisione in contesti di business/progetti e controllarne gli accessi almeno in modifica/scrittura.

*RT37**

Il catalogo dati della data platform deve offrire interfacce sia utente che programmatiche per la consultazione e l'integrazione.

*RT38**

Il catalogo dati della data platform deve presentare un'interfaccia che renda facile la ricerca e il reperimento delle informazioni. Deve offrire strumenti per la gestione collaborativa del patrimonio informativo.

*RT39**

Per ogni componente dell'architettura della data platform e' necessario un supporto tecnico in fase di avviamento e mantenimento. Qualora se ne riscontrasse la necessità, dovranno essere offerti corsi di formazione specifici.

RT40

Le componenti della data platform per l'orchestrazione, l'estrazione e il caricamento del dato dovrebbero offrire interfacce visuali per il controllo, la gestione e lo sviluppo delle pipeline.

RT41

Le componenti della data platform per l'orchestrazione dovrebbero essere in grado di automatizzare lo scheduling dei task e dei job che vengono eseguiti sui motori di processamento.

RT42

La data platform dovrebbe permettere la definizione di pipeline con un paradigma orientato al controllo dei flussi, permettendo di fare routing sui dati attraverso attributi e metadati.

RT43

La data platform dovrebbe offrire meccanismi avanzati di protezione, tipici del controllo dei flussi, da fonti che non si comportano secondo le aspettative.

RT44

La data platform dovrebbe permettere di versionare le pipeline e le automazioni del flusso di gestione del ciclo di vita del dato.

RT45

La data platform dovrebbe offrire la possibilità di definire politiche di gestione di perdita del dato e strumenti che ne permettano l'implementazione.

RT46

La data platform dovrebbe permettere di definire un livello di conferma di delivery del dato che possa arrivare alla singola riga o al singolo file.

RT47

La data platform dovrebbe offrire un ambiente di analisi, esplorazione e processamento capace di interpretare ed eseguire codice in linguaggio Python.

RT48

La data platform dovrebbe offrire un ambiente dedicato all'esplorazione interattiva del dato con un'interfaccia utente in stile notebook.

RT48.1

L'ambiente dedicato all'esplorazione interattiva del dato dovrebbe fornire la capacità di integrare analisi geospaziali e visualizzazioni cartografiche.

RT49

La data platform dovrebbe essere in grado di eseguire job e task di analisi in modalità parallela, distribuendo il carico in più entità di calcolo indipendenti in maniera trasparente rispetto allo sviluppo e agli utenti.

RT50

La data platform dovrebbe offrire meccanismi di single-sign-on sul maggior numero di componenti di sviluppo, implementazione e integrazione.

RT51

La data platform dovrebbe prevedere funzionalità per l'auditing delle operazioni eseguite dagli utenti sui dataset.

RT51.1

La data platform dovrebbe permettere di estrarre metadati dai processi di gestione del ciclo di vita del dato.

RT52

la data platform dovrebbe offrire la possibilità di tracciare il lineage del dato, e di mostrarne l'albero delle dipendenze.

RT53

La data platform dovrebbe prevedere la possibilità di criptare file a livello di memorizzazione fisica.

RT54

La data platform dovrebbe permettere di definire le politiche di estrazione e disponibilità del dato (best effort, only once, exactly once, etc...)

RT55

La data platform dovrebbe offrire funzionalità per la memorizzazione di schemi e modelli permettendo a più versioni di coesistere contemporaneamente in maniera indipendente e versionati nel tempo.

RT56

La data platform dovrebbe permettere la definizione e il mantenimento di metadati relativi alle norme vigenti in materia di data protection, privacy e quality (data regulation).

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

3 - Sintesi dell'architettura

La data platform è un'architettura software modulare a componenti che permette di:

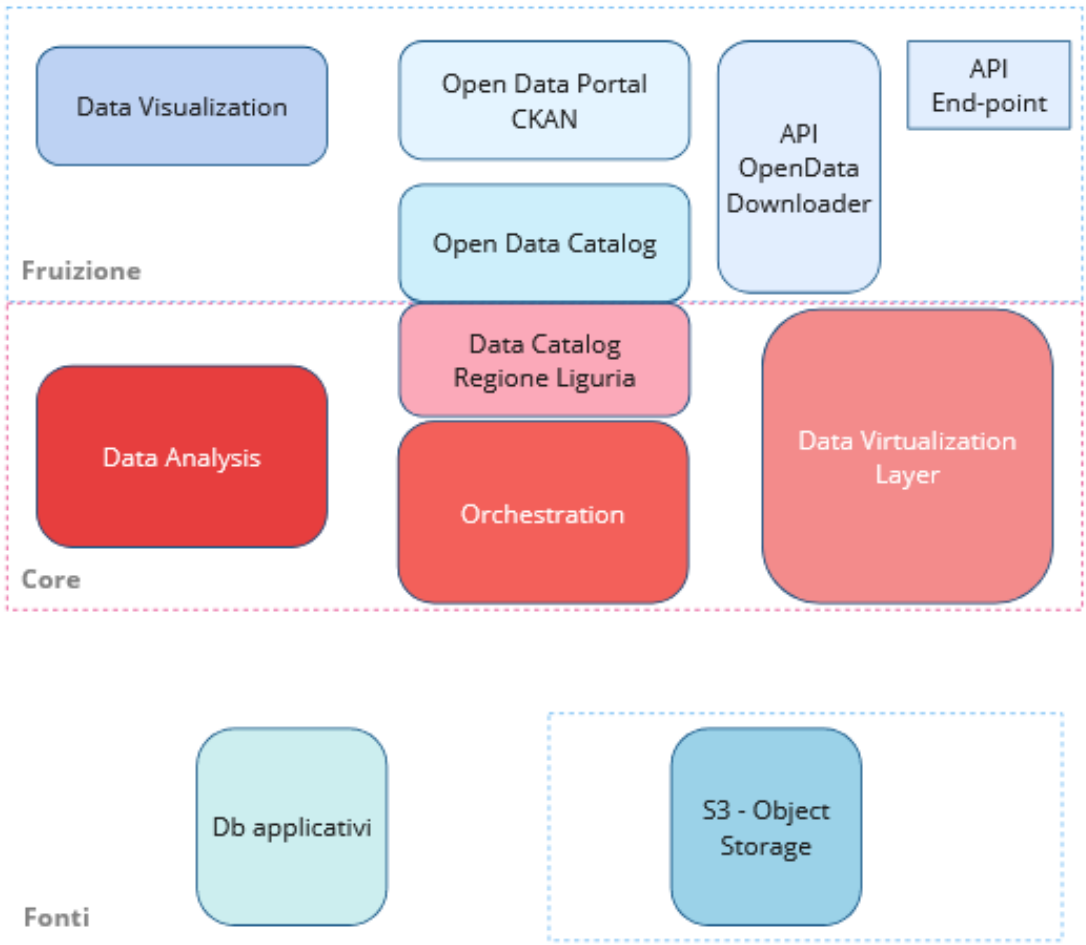
- fungere da hub centrale per unificare diverse fonti di dati, garantendo accesso e analisi facilitati.
- abilitare l'implementazione di politiche per la gestione responsabile del ciclo di dati, assicurando conformità, sicurezza e privacy.
- assicurare l'accuratezza e l'affidabilità dei dati attraverso processi di validazione e standardizzazione.
- permettere ai cittadini di accedere facilmente ai dati, promuovendo trasparenza e partecipazione.
- offrire strumenti di analisi e visualizzazione per estrarre insight utili e produrre report comprensibili
- adattarsi all'evoluzione delle esigenze e delle tecnologie, assicurando che la piattaforma rimanga efficace e rilevante nel tempo.

L'architettura della data platform si articola in tre livelli logici di funzionalità:

- Fonti: sono comprese le fonti dato sia esterne per l'alimentazione del sistema che lo "spazio di memorizzazione" interno alla data platform, implementato con una piattaforma Object Storage S3

- Core: sono comprese tutte le funzionalità principali che abilitano lo sviluppo di reportistica e che rendono disponibili i dati ad altri sistemi

- Fruizione: fa rifermento a tutte le componenti e gli strumenti che sono rivolte agli utenti e sistemi finali



Le componenti di alto livello sono:

API End-point
Possibilità di esporre API protette attraverso il nostro identity provider in compliance con il sistema PDND

API OpenData Downloader
Un sistema che abiliti i cittadini a reperire, costruire e scaricare i dataset contenuti nel catalogo, in diversi formati “human-readable” e “machine-readable” , e attraverso servizi API

Open Data Portal CKAN
Integrare un front-office per la consultazione pubblica del catalogo da parte dei cittadini, imprese e ricercatori

Data Visualization
Una piattaforma open per la fruizione di visualizzazioni e grafici sui dataset prodotti e mantenuti dalla Data Platform

Open Data Catalog
Dotare Regione Liguria di un catalogo dati open interoperabile con i processi di Data Governance e Quality nazionali ed europei

Data Catalog Regione Liguria
Dotare Regione Liguria di un catalogo dati unificato per il patrimonio dati interoperabile con i sistemi, i processi di Data Governance e Quality interni

Data Analysis
Una architettura software che supporti tutte le fasi della preparazione, esplorazione e dell'analisi del dato per estrarre valore aggiunto che alimenti il catalogo in ottica di supporto alla decisione e di distribuzione open data

Orchestration
"Una piattaforma software che instrada, mantiene aggiornati e orchestra i flussi dato e i dataset che si muovono sia all'interno che all'esterno della Data Platform

Data Virtualization Layer
Uno strato di virtualizzazione per omogeneizzare e standardizzare l'accesso alle diverse fonti dato e per l'accelerazione in fase di restituzione del dato"

S3 - Object Storage
Un supporto hardware/software di persistenza fisica per l'accumulo e la conservazione dei dataset rendendoli accessibili attraverso interfacce standard a tutte le componenti della Data Platform

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

4 - Descrizione dell’architettura

La definizione e successivo dispiegamento dell’architettura è la prima attività prevista e rappresenta la condizione necessaria per l’implementazione dei vari casi applicativi che verranno sviluppati nel corso dello svolgimento del progetto,

Di seguito verranno dettagliate le macro-componenti introdotte nel capitolo precedente e loro interazioni, evidenziando e motivando le scelte di tipo tecnologico o di processo di business.

4.1 - Descrizione dell'architettura

I macro-componenti che costituiscono l’architettura vengono descritti in gruppi di contesto che permettano di far emergere le modalità di interoperabilità dei componenti che coprono funzionalità complementari.
Alcuni degli strumenti e delle componenti sono coinvolti in diversi blocchi logici, e quindi gli aspetti associati sono illustrati nei capitoli di riferimento.

Di seguito e’ presente una lista delle componenti coinvolte in ogni capitolo, con riferimento alla descrizione specifica presente nel capitolo 5, per permettere di consultare la descrizione di dettaglio in maniera piu’ semplice.

- 4.2 – Object Storage e Data Virtualization
    - MinIO: capitolo 5.1
    - Iceberg: capitolo 5.2
    - Project Nessie: capitolo 5.3
    - Dremio: capitolo 5.4
- 4.3 – Orchestration
    - Nifi: capitolo 5.5
    - Airflow: capitolo 5.5
    - Mage: capitolo 5.5
    - Kafka: capitolo 5.6

  - dbt: capitolo 5.7
  - Spark: capitolo 5.8
  - DataHub: capitolo 5.10
  - Open MetaData: capitolo 5.10

- 4.4 - Data Analysis

  - Spark: capitolo 5.8
  - Dremio: capitolo 5.4
  - dbt: capitolo 5.7
  - Jupyter: capitolo 5.9
  - Zeppelin: capitolo 5.9

- 4.5 - Catalogo Regione Liguria e Catalogo Open Data

  - Open MetaData: capitolo 5.10
  - Data Hub: capitolo 5.10

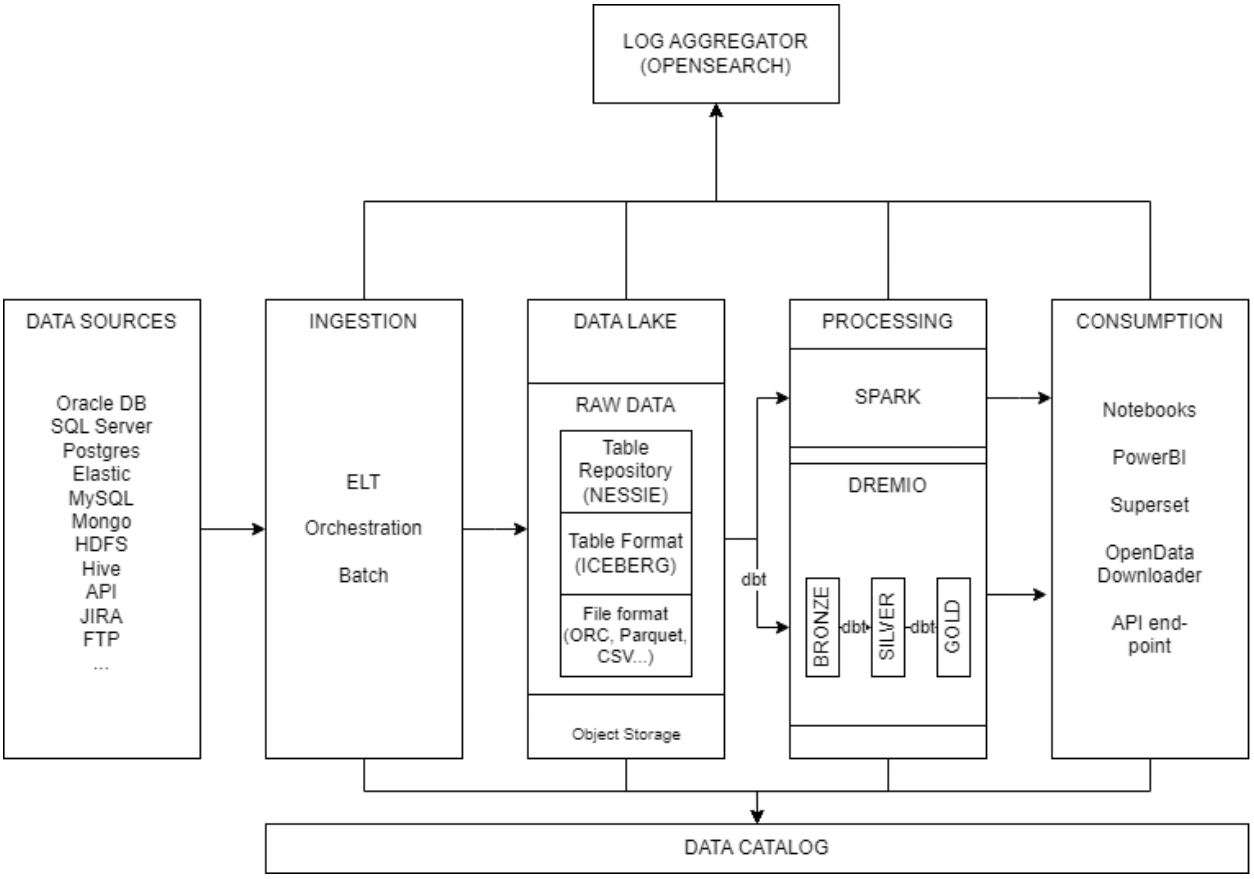
- 4.6 - Fruizione: Integrazioni Cartografiche, Portale CKAN, Open Data Downloader e API end-point

  - SuperSet: capitolo 5.13
  - Componenti Cartografici: capitolo 5.12
  - Componente Downloader e Api REST: capitolo 5.14

- 4.7 - Logging e monitoraggio

  - Open Search: capitolo 5.11

Segue un diagramma di alto livello dell'architettura, che mostra le fasi in successione e indica alcuni dei prodotti coinvolti nel ciclo di vita.



Nello schema soprastante possiamo vedere come il dato si muove:

- Partendo dalle fonti, che possono essere di tipologia e provenienza diverse
- Passa per una fase di ingestion che estrae il dato dalle fonti
- Viene caricato sul data lake, in diverse modalità
- Attraversa varie fasi di processamento e trasformazione
- Aumentando di valore e di livello di aggragazione (bronze, silver e gold)
- È pronto ad essere consumato dai sistemi di fruizione
- Tutte le operazioni eseguite in ciascuna fase sono:
    - Metadate e catalogate
    - Monitarate e controllate

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

4.2 - Componente di Object Storage e Data Virtualization

Il supporto alla memorizzazione dei dataset sarà implementato attraverso un sistema di archiviazione ad oggetti (object storage) che deve essere compatibile con il protocollo S3. La scelta per rispondere alla necessità ricade su una configurazione cluster di MinIO, che può essere installato su nodi commodity, ma con supporto disco di tipo flash. L'attuale dimensionamento stimato costa di 4 host, con 16 cpu, 32 GB di memoria e 4 drive ad host da 1TB ciascuno, per un totale utilizzabile di circa 32TB tenendo in considerazione i meccanismi di checksum e le repliche necessarie a garantire la consistenza del dato.

Questo nucleo rappresenta il primo strato di astrazione del data lake, il luogo dove i dataset risiedono fisicamente in formati tabellari, quali parquet e iceberg, accessibili attraverso le API tipiche del protocollo S3. Per avere aderenza ai requisiti, è necessario estendere le funzionalità di accesso al dato e di versionamento.
Il controllo delle versioni viene gestito in maniera git-like, attraverso l'interazione tra Project Nessie, che permette di costruire un sistema a repository sui delta incrementali dei cambiamenti che Iceberg traccia. Inoltre, MinIO e' in grado di registrare e scatenare eventi a fronte di cambiamenti sui dataset fisici o a partire da interazioni delle componenti applicative; questo facilita l’interoperabilità, la governabilità dei processi e la manutenibilità.

Il dimensionamento stimato per il deploy iniziale di Project Nessie sarà costituito da un unico nodo, con 2 cpu 8 GB di memoria e 50 GB di storage.

Le componenti descritte precedentemente compongono la parte essenziale del data lake, questo fornisce un protocollo di accesso, un sistema di gestione delle versioni e dei cambiamenti dei dati al suo interno.
L'interfaccia S3 presenta dei limiti nelle possibilità di accesso e interrogazione, in quanto espone semplicemente un riferimento al file fisico di memorizzazione, mentre non offre alcuna interfaccia interrogabile programmaticamente che supporti i requisiti per le funzionalità analitiche da implementare. Per questo motivo è necessario inserire un nuovo livello di astrazione, che si allontani dalla rappresentazione fisica del dataset e che presenti interfacce di più alto livello che permettano di interpretare le informazioni e le strutture presenti sul supporto fisico di memorizzazione.

La modalità di interrogazione dei dati più diffusa è quella attraverso il linguaggio SQL, solitamente limitato a fonti relazionali e quindi non applicabile a dati non strutturati o semi-strutturati come nel nostro caso. L'ubiquità di SQL come strumento di interrogazione e analisi è una funzionalità imprescindibile per garantire standardizzazione, continuità delle interazioni utente/macchina e semplicità di utilizzo. Per soddisfare questa necessità, è necessario integrare una componente che sia in grado di astrarre il formato sottostante per fornire una rappresentazione conforme ad un modello compatibile con SQL. Questo può essere fatto con un oggetto che implementi almeno tutte le funzionalità solitamente associate allo standard ANSI SQL, e che estenda le capacità per formati non supportati.

Per coprire le funzionalità di astrazione sopra descritte viene introdotto nel disegno un ulteriore componente individuato in Dremio, che si occupa di fornire una interfaccia SQL-complaint da e verso le altre componenti dell'architettura. La sua responsabilità principale è quella di esporre i

dataset memorizzati nel datalake attraverso il suo motore di elaborazione SQL agnostico rispetto alla fonte del dato, sia per quanto riguarda i formati che la provenienza.

Questa capacità di interrogare nello stesso modo fonti diverse permette l'integrazione di dataset disomogenei per contesto, formato, e provenienza data un'intersezione semantica o di contesto delle informazioni contenute. I dataset, derivati da queste integrazioni, possono essere raggruppati in livelli semantici gerarchici all'aumentare progressivo del grado di astrazione dalle fonti raw, incrementandone il valore per il business e per le applicazioni.

Dremio offre la possibilità di mediare l'accesso alla fonte originale, nascondendo il dettaglio implementativo della soluzione sottostante, garantendo maggiore uniformità nel reperire il dato. Inoltre, se necessario, invece di fungere da semplice middleware per l'accesso, può sostituire il motore di elaborazione delle fonti originali facendosi carico direttamente delle elaborazioni, alleggerendo il carico sui sistemi di gestione da cui provengono i dati.

Le elaborazioni eseguite attraverso il motore della componente vengono rese disponibili direttamente da una cache in-memory, velocizzando l'accesso al dato da parte delle altre componenti dell'architettura. Per garantire la compatibilità di Dremio con i meccanismi di versioning resi possibili da Project Nessie e Iceberg, dobbiamo promuovere un Nessie Catalog al tracciamento dei datasource. In questo modo possiamo definire e gestire diverse versioni/edizioni dei dataset.

Infine, Dremio implementa funzionalità di security a livello di riga oppure di colonna, permettendo di controllare l'accesso granulare al dataset. In questo modo, possiamo utilizzarlo come livello di accesso unico per le applicazioni di reportistica, semplificando notevolmente le fasi di integrazione.

La configurazione di deploy individuata per il cluster Dremio prevede una divisione dei nodi in master e slave. Il coordinator funge da master e avrà a disposizione 16 cpu, 32GB di memoria e 200 GB di storage e si occuperà di gestire lo scheduling dei job e dei task degli executor. Le componenti slave avranno una numerosità di partenza pari a 3 nodi, e saranno composte da istanze con 16 cpu, 128 GB di memoria, e 300 GB di spazio di archiviazione. Le possibilità di scaling vanno prevalentemente in direzione orizzontale, quindi tramite l'aggiunta di nuovi nodi executor al cluster.

Alla luce di quanto presentato, con l'aggiunta di Dremio alle componenti precedentemente descritte, vengono rispettati i requisiti per l'implementazione dello strato di data virtualization.

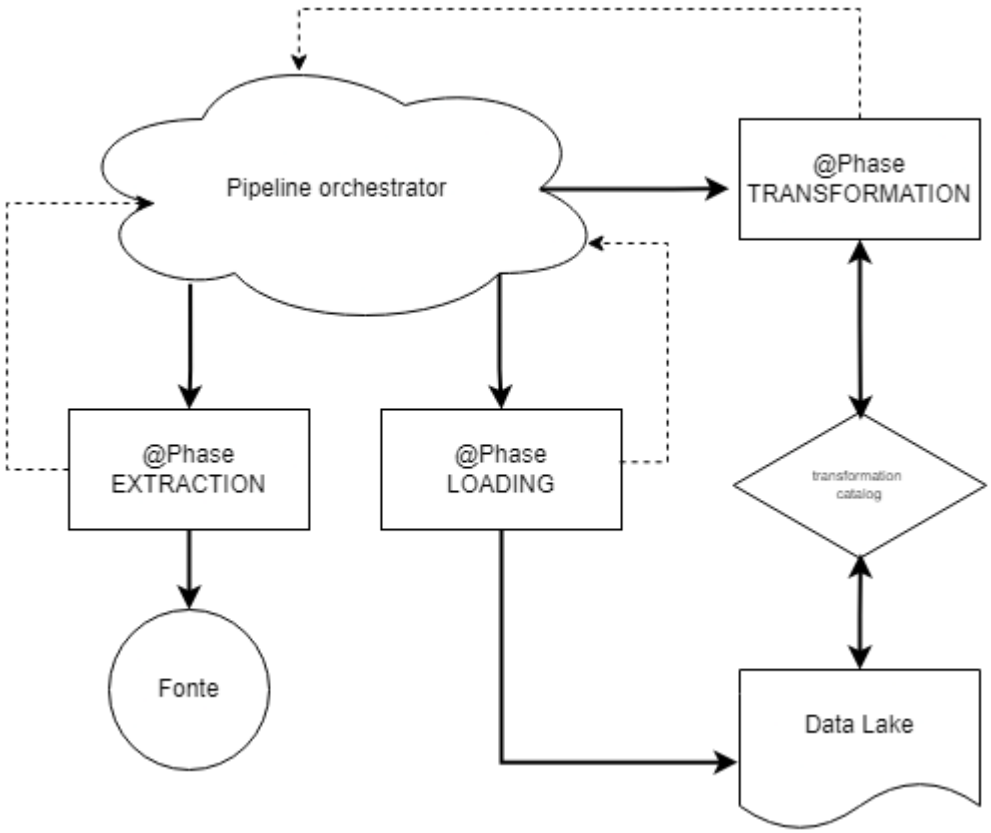
Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

4.3 - Componente di Orchestration

Al fine di approvvigionare il dato in maniera automatica e costante, è necessario avere una o più componenti in grado di gestire flussi provenienti dalle fonti.

Avere il dato al proprio interno abilita la piattaforma ad utilizzare tutte le sue componenti al meglio delle capacità, evitando di fare affidamento su tecnologie esterne che possano interferire con il processo. Per questo motivo, nella fase di estrazione, è buona pratica importare il dato in formato raw, secondo il pattern ELT (Extract, Load, Transform). Grazie a questo approccio, le differenze tra i dati delle applicazioni gestionali e quelle alla base dei processi di analytics saranno limitate o assenti, garantendo che le trasformazioni siano fatte una volta soltanto. È così possibile garantire una maggiore qualità del dato e un minor impatto sulla gestione e sul mantenimento del patrimonio informativo. Inoltre, in questo modo si alleggeriscono i sistemi gestionali dai carichi analitici e si possono sfruttare appieno la scalabilità e le capacità elaborative di strumenti dedicati e ingegnerizzati per questo scopo all'interno della piattaforma.

A questo punto l'on-boarding del dato all'interno della data platform è terminato e sarà quindi disponibile per successive trasformazioni ed elaborazioni. Anche queste fasi cadono sotto la responsabilità dell’orchestrator, che è in grado di triggerare, monitorare e schedulare le altre componenti worker dell'architettura, che sono incaricate di portare a termine le singole fasi di elaborazione mantenendone la consistenza.



Come si evince dallo schema soprastante il ruolo dell'orchestrator è quello di coordinare le varie fasi del ciclo di vita del dato. A partire dalla fonte da cui viene estratto il dato per poi essere caricato sul datalake, attraverso una delle interfacce disponibili. Quindi il dato viene elaborato secondo le necessità di business, per iterazioni successive di trasformazione, per ognuna delle quali viene tracciata la storia delle modifiche.

Questo processo potrebbe avvenire all'interno dell'orchestrator stesso, tuttavia saranno istanziate risorse di elaborazione remote su altri host per separare il control plane dall'esecuzione delle singole fasi mantenendo così l'orchestrator disponibile e responsivo.

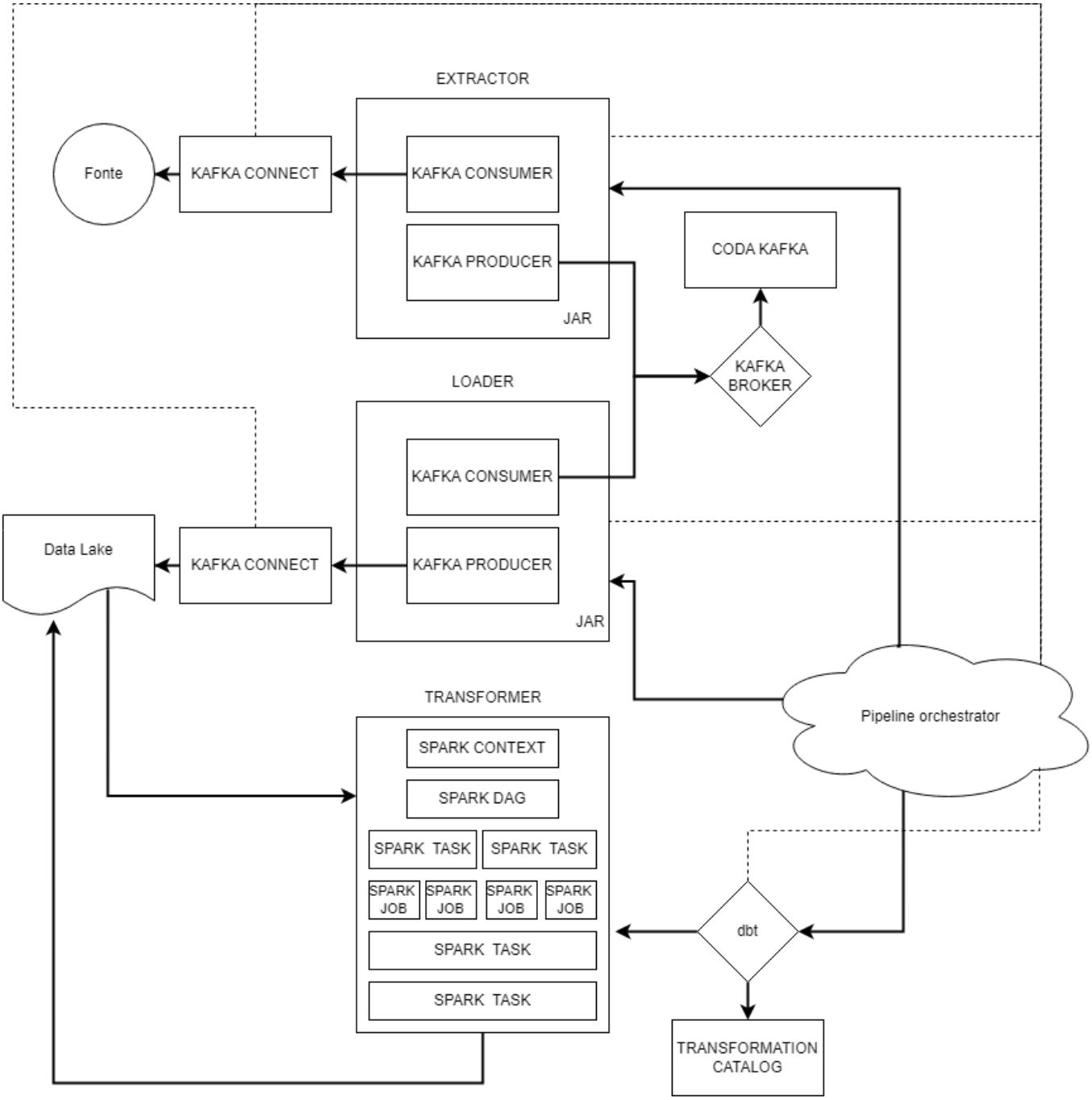
Un'altra responsabilità dell'orchestrator è quella di monitorare il processo in ogni sua fase. Ogni volta che avanza il processo, viene registrato l'esito dell'operazione per monitorare e controllare lo stato della pipeline. In caso di comportamenti anomali, è possibile attivare alert e notifiche per permettere un'azione tempestiva precedentemente definita oppure eventualmente se richiesto un intervento da parte di un amministratore/sviluppatore del

sistema. Queste funzionalità sono rese disponibili attraverso un'interfaccia web, che permette facile accesso e immediatezza nella visione dello stato del sistema.

Ciascuna fase è caratterizzata da una responsabilità specifica e unica e può essere implementata come un'integrazione con altre componenti dedicate all'interno della data platform, o in casi di alta customizzazione attraverso script ad-hoc che possono interagire a loro volta con diverse componenti.

Ogni fase del processo di gestione del dato, deve essere in grado produrre e fornire i metadati necessari al catalogo dati per costruire il lineage e lo schema dati aggiornato. Le modalità di alimentazione del catalogo sono da definire al momento della scelta degli strumenti. Si intravedono due strade, per lo più equivalenti nei risultati. La prima prevede un meccanismo di sincronizzazione automatica dei metadati, mentre la seconda utilizza una modalità push da definire all'interno della fase di riferimento.

Viene condiviso l’attuale schema di gestione dell'intero ciclo di vita del dato; a partire dalla fonte e attraverso tutte le successive fasi viene illustrata l'interazione delle componenti con l’orchestrator.


EXTRACTOR
Fonte
KAFKA CONNECT
KAFKA CONSUMER
KAFKA PRODUCER
JAR
CODA KAFKA
KAFKA BROKER
LOADER
KAFKA CONSUMER
Data Lake
KAFKA CONNECT
KAFKA PRODUCER
JAR
TRANSFORMER
Pipeline orchestrator
SPARK CONTEXT
SPARK DAG
SPARK TASK
SPARK TASK
SPARK JOB
SPARK JOB
SPARK JOB
SPARK JOB
SPARK TASK
SPARK TASK
dbt
TRANSFORMATION CATALOG

Il punto di partenza della fase di estrazione vede coinvolta la fonte, ad esempio un database relazionale di un'applicazione gestionale, che viene agganciato attraverso un Kafka Connect, che media le interazioni con il sistema RDBMS. A questo punto è possibile alimentare una coda Kafka attraverso il pattern consumer/producer in cui viene estratto il dato dalla fonte riga per riga e successivamente instradato all'interno di una opportuna struttura dati. Le interazioni sulle code sono mediate da Broker che permettono una lettura non sequenziale e parallela dei dati. Lo stesso pattern viene impiegato per ricostruire le porzioni del dato e caricarli sul Data Lake. Ciascuna di queste fasi viene scatenata e monitorata dall'orchestrator, che si fa carico di gestire il corretto funzionamento del processo globale.

La fase di transformation inizia quando tutto il dataset è caricato correttamente sul Data Lake e l'orchestratore fa partire le trasformazioni necessarie, recuperandole da “dbt” che funge da repository di tutte le trasformazioni. Quindi le elaborazioni vengono schedulate sul cluster Spark e divise in task e job che verranno eseguiti con il massimo grado di parallelizzazione permessa dalle risorse disponibili di computazione e memoria. Infine, il dataset trasformato viene scritto sul Data Lake e diventa disponibile per i servizi di reportistica e per le applicazioni.

Il dimensionamento della componente di orchestrazione dipenderà dal livello di multi tenancy e dalla configurazione del deploy. A fronte dell'esperienza maturata negli anni con componenti similari per obiettivi e funzionalità riteniamo necessario contemplare una divisione per contesti delle risorse e dell'accesso all'orchestrator, per garantire una governance più oculata e isolata dei processi.

Per quanto concerne la configurazione dell'orchestrator rimandiamo alla descrizione puntuale della componente [inserire riferimento capitolo]

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

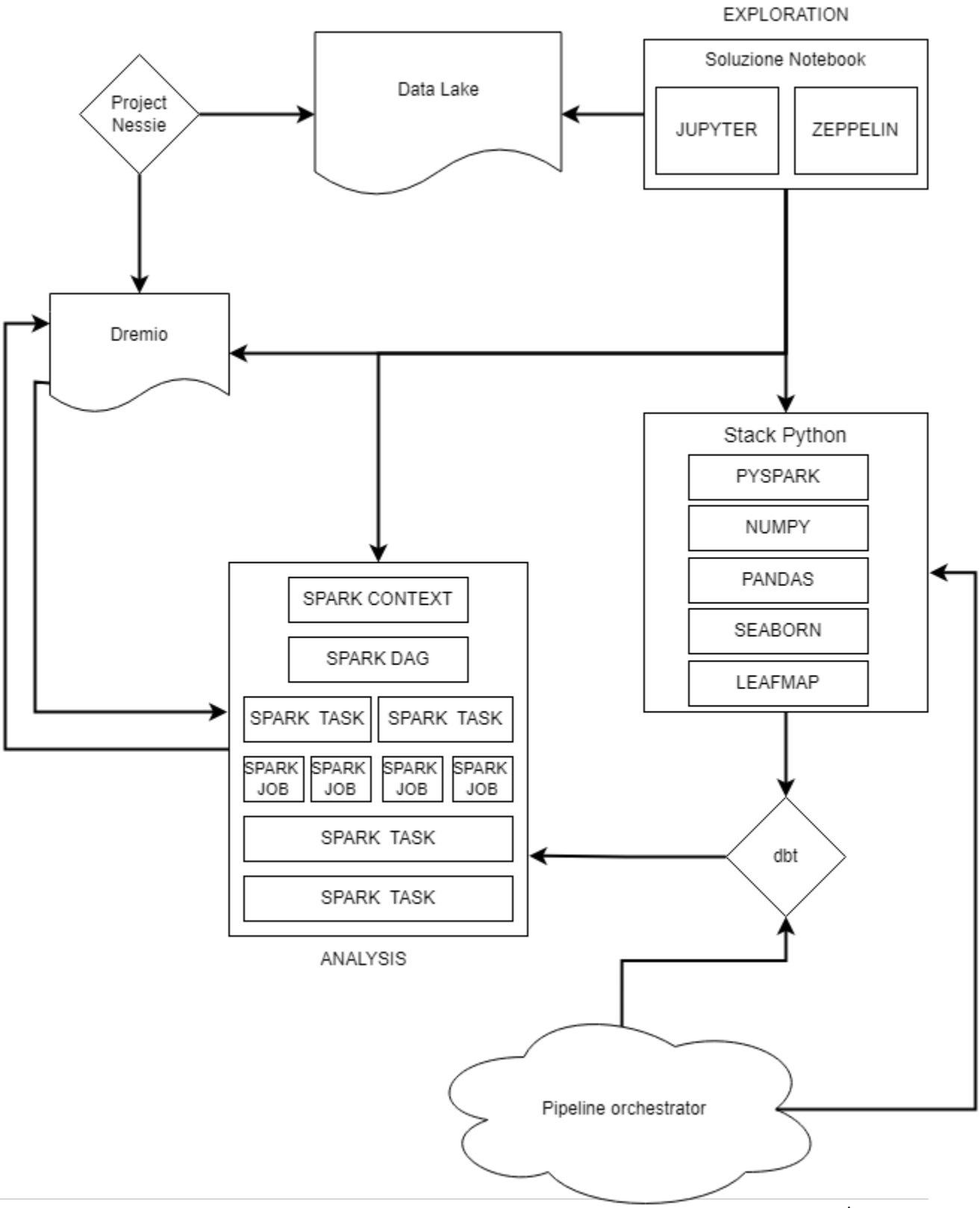
4.4 - Componente di Data Analysis

La componente di Analysis della data platform si occupa di implementare le funzionalità di analisi, trasformazione dei dati, esplorazione e interfacce per la restituzione.

All'interno di questo layer, Dremio si occupa di rendere accessibile il dato alle altre componenti attraverso le interfacce (SQL, Arrow Flight, REST...) che mette a disposizione. Il dato fisico è potenzialmente accessibile tramite Api REST con il protocollo S3, ma e' preferibile non utilizzare questa modalità, tranne che per l'esplorazione del dato (data exploration), cioè nella fase in cui si cerca di capire la semantica e le peculiarità del dato.

In fase di trasformazione e analisi, l'accesso alle interfacce esposte da Dremio viene mediato da “dbt”, che funge da unico repository di tutte le trasformazioni per permetterne la catalogazione, il testing e il monitoraggio. Seguendo questo pattern sarà possibile garantire un maggior controllo dei processi batch e di trasformazione del dato, nonché' di aumentarne le capacità di validazione e di governance. Infine, si registrerà un guadagno anche in termini di flessibilità e robustezza in fase di versionamento, in quanto è possibile versionare e storicizzare le trasformazioni catalogate con “dbt”, facilitando la recovery e il roll back dei dati.

Sarà quindi possibile offrire versioni diverse dei dataset allo stesso tempo, semplicemente definendole con trasformazioni e Virtual Data Set (VDS) nell'interazione di “dbt” con Dremio, come necessario per adempiere alle linee guida AGID sugli open data. Le VDS saranno organizzate e gestite attraverso una gerarchia semantica che rispecchi il valore aggiunto presente nei dataset.


EXPLORATION
Soluzione Notebook
JUPYTER
ZEPPELIN
Data Lake
Project Nessie
Dremio
Stack Python
PYSPARK
NUMPY
PANDAS
SEABORN
LEAFMAP
SPARK CONTEXT
SPARK DAG
SPARK TASK
SPARK TASK
SPARK JOB
SPARK JOB
SPARK JOB
SPARK JOB
SPARK TASK
SPARK TASK
ANALYSIS
dbt
Pipeline orchestrator

Per tutte le trasformazioni ed elaborazioni che richiedono un parallelismo o tempi di risposta più rapidi, è necessario integrare nel processo di analisi anche Spark.

Spark accederà al dato attraverso Dremio, il tutto sotto il controllo di “dbt”. Spark è un motore di elaborazione alternativo rispetto a quello integrato in Dremio.

Per maggiori informazioni sulle sue funzionalità e sull’operativita’, si rimanda alla descrizione della componente al capitolo xxx.

Un'altra alternativa per l'analisi e le trasformazioni è lo stack Python, che può eventualmente integrare anche Spark, attraverso una SDK definita nel linguaggio. Sicuramente il principale utilizzo di Python all'interno di questa componente è in fase di esplorazione (data exploration), in quanto garantisce un maggior livello di flessibilità. A supporto dell'esplorazione, verranno utilizzati Jupyter Notebooks o Zeppelin come interfaccia web disponibile agli utenti che svilupperanno soluzioni in questo contesto operativo.

Ciascuna di queste componenti è in grado di comunicare con Dremio per integrare VDS in maniera tale da semplificare l'accesso a tutte le altre componenti e sistemi di fruizione esterni, offrendo allo scopo una interfaccia SQL virtuale. Qualora le applicazioni gestionali necessitino di accedere ai dati elaborati nella data platform, sono previste modalità di write-back sul database di origine per garantire maggiore integrazione e continuità.

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

4.5 – Componenti Catalogo Regione Liguria e Catalogo Open Data

Con l'obiettivo di supportare processi di governance del dato e di controllo della qualità, viene inserita nel core dell'architettura un sistema di catalogazione dei dati che offre numerose funzionalità quali:

- Data Discovery
- Data Observability
- Data Lineage
- Data Quality
    - Reliability
    - Completeness
    - Timeliness
    - Consistency
    - Veracity
- Data Governance
    - Ownership
    - Discoverability
    - Compliancy

Per maggiori dettagli consultare la descrizione di dettaglio della componente.

Inoltre, questa componente sarà integrata con l'attuale catalogo open data, fornendo i metadati necessari alla compliance con gli standard di raccomandazione AGID per la pubblicazione dei dati aperti, andando a coprire delle mancanze segnalate nel modello DCAT-AP.

4.6 - Componenti di fruizione: Integrazioni Cartografiche, Portale CKAN, Open Data Downloader e API end-point

Per le componenti cartografiche, downloader e API Open Data end-point, si rimanda alla descrizione delle singole componenti al capitolo, dove è descritto il ruolo che in seguito allo sviluppo e all'integrazione, ricopriranno all'interno dell'architettura.

Per quanto concerne il portale CKAN, è previsto un aggiornamento della componente, e un'ipotesi di semplificazione dell'architettura di questa, per ridurre la complessità di mantenimento e l'impatto infrastrutturale.

Le capacità di reportistica verranno estese dall'introduzione di Superset in affiancamento a Power Bi. La necessità viene dalle richieste sempre maggiori di integrazione di reportistica all'interno di applicazioni gestionali, che al momento sono limitate con la versione on-prem di Power Bi. Inoltre, è desiderabile diminuire la dipendenza dalla soluzione Microsoft, in quanto i trend di mercato e le strategie, portano sempre di più questi prodotti verso un'offerta cloud-only, progressivamente limitando le funzionalità delle versioni on-prem, di fatto forzando la migrazione. Anche adottando soluzioni open source, il problema dell'approccio cloud-only, persiste, seppur con costi più contenuti.

In ogni caso avere sotto continuo controllo il mercato delle soluzioni open mitigherà il rischio di Lock-in, permettendoci di adattare la nostra offerta.

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

4.7 – Componenti di Logging e Monitoraggio

Dato il grande numero di componenti eterogenee sviluppate e integrate all'interno dell'architettura ed il numero di nodi, diventa prioritario avere un unico luogo dove archiviare i log applicativi e di sistema per facilitare le operazioni di debug e monitoraggio complessivo.

La soluzione prevederà un approccio ad agenti, disseminati sui nodi periferici, che recupereranno e invieranno i log verso un cluster centrale di Open Search, che si occuperà della conservazione, consultazione ed analisi, mostrando alerts e grafici sull'andamento di metriche definite.

Il volume dei log prodotti impone la definizione di politiche di retention e l'implementazione di flussi di aggregazione per la conservazione delle informazioni di interesse distillate.

I flussi dei log di monitoraggio potranno essere usati come benchmark e per rivalutare processi e dimensionamenti.

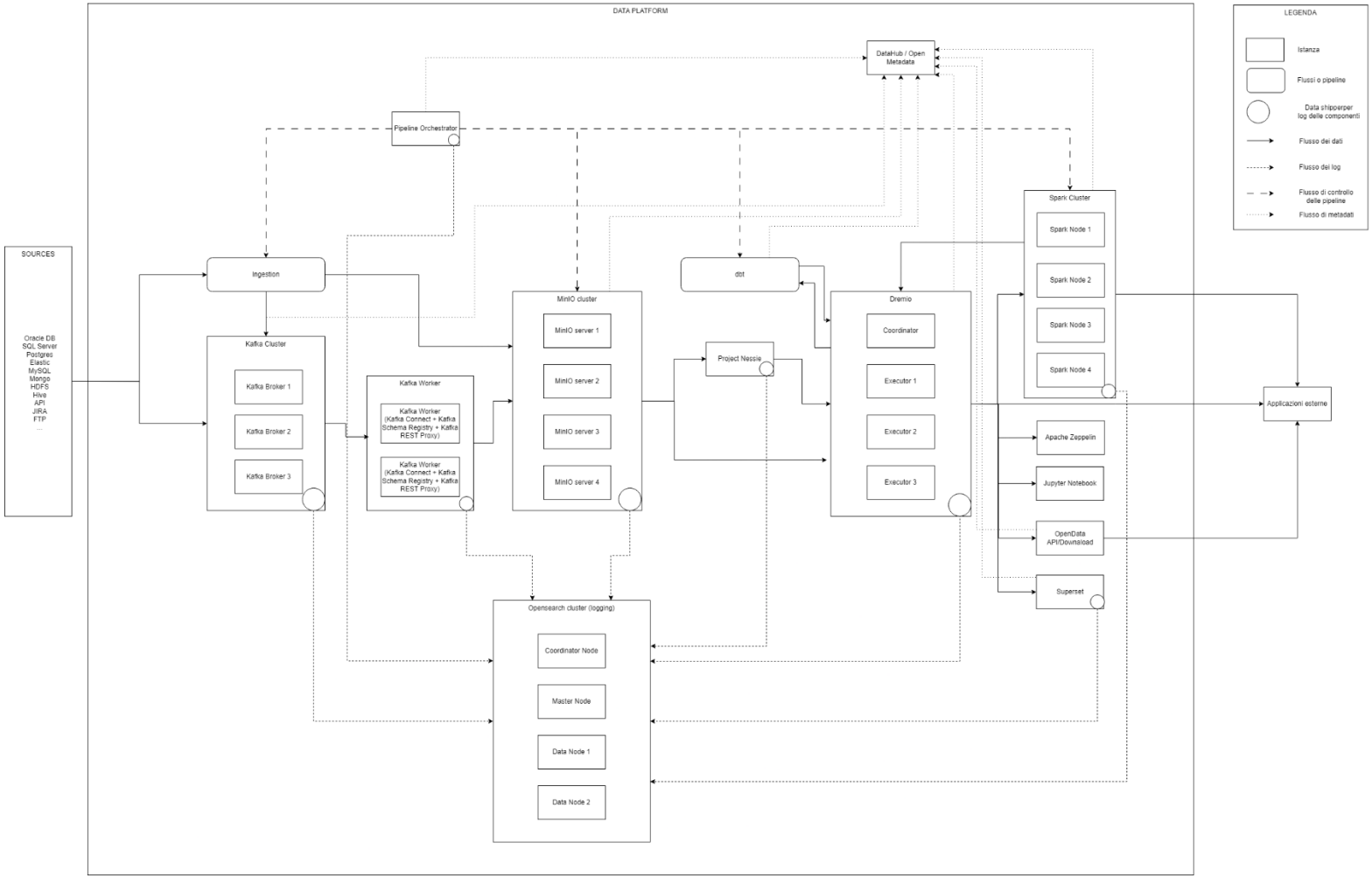
Per maggiori dettagli confrontare la componente di riferimento al capitolo 5.11.

4.8 – Diagramma generale

Segue il diagramma generale dell'architettura, in cui viene mostrata e sintetizzata la configurazione completa descritta nei capitoli precedent

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422



Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5 - Descrizione dei componenti

In questo capitolo vengono presentate e descritte nel dettaglio, tutte le componenti di interesse rispetto all'architettura.

Verranno introdotte da un'introduzione generale alla tipologia di soluzione e di funzionalità che potranno ricoprire all'interno del sistema, per poi continuare con un drill-down sulle specifiche tecniche del componente.

Saranno presentati dei grafici a corredo per illustrare determinati comportamenti, meccaniche o funzioni notevoli per l'implementazione e il dispiegamento.

5.1 - Componente MinIO

Object storage

All'interno di un'infrastruttura dati di tipo Data Lake, l'object storage si occupa di gestire e memorizzare i dati. Nel nostro caso, si tratta del punto di arrivo dei dati dopo le procedure di loading, e la posizione da cui vengono letti i dati da Dremio, prima di effettuare le trasformazioni richieste.

All'interno dell'ecosistema del Data Lake, l'Object Storage contribuisce a:

1. Memorizzazione scalabile e flessibile: L'object storage consente di memorizzare enormi quantità di dati in modo scalabile e flessibile. È in grado di ospitare sia dati strutturati che non strutturati.
2. Accesso rapido e distribuito: È progettato per consentire l'accesso rapido e distribuito ai dati. Gli oggetti memorizzati sono indirizzabili tramite URL univoci, il che consente un recupero efficiente dei dati da qualsiasi punto dell'architettura senza dipendere da una struttura gerarchica rigida.
3. Durabilità e ridondanza: Solitamente, i servizi di object storage offrono alti livelli di durabilità e ridondanza. I dati vengono replicati su più dispositivi e posizioni geografiche per garantire la loro integrità e disponibilità anche in caso di guasti hardware o errori.
4. Compatibilità con strumenti di analisi e elaborazione: L'object storage è progettato per essere compatibile con una vasta gamma di strumenti di analisi, elaborazione e data science. Questa caratteristica consente agli utenti di eseguire analisi, machine learning e altri processi direttamente sui dati memorizzati senza spostarli su altre piattaforme.

MinIO

MinIO è un object storage che fornisce compatibilità alle API Amazon Web Services S3, e supporta tutte le sue funzionalità core. MinIO è nato per essere installato su Kubernetes, tuttavia può essere installato in ogni modo: reti pubbliche o private, bare metal, in ambienti orchestrati o infrastrutture edge.

Funzionalità principali

Minio offre una serie di feature core, le quali sono:

1. Scalabilità: Minio è un sistema distribuito che può essere scalato orizzontalmente aggiungendo nodi.
2. Disponibilità: Al fine di garantire l'alta affidabilità, Minio replica i dati su più nodi, in modo tale da persistere i dati, anche in caso di perdita di uno o più nodi.
3. Sicurezza: Vengono offerte una serie di funzionalità sulla sicurezza, tra cui autenticazione, controllo degli accessi e crittografia dei dati salvati
4. Compatibilità: Viene fornita la compatibilità con il protocollo S3, il che lo rende facile da usare con applicazioni che già utilizzano questa tipologia di protocollo.

Object Immutability

MinIO supporta il blocco, la conservazione, governance e conformità degli oggetti salvati. Il blocco degli oggetto può essere utilizzato in combinazione con il versioning in modo tale da garantire l'immutabilità dei dati ed eliminare il rischio di manomissione o distruzione di questi ultimi.

Le regole di retention garantiscono che ogni oggetto sia protetto per un determinato periodo di tempo. La retention può essere impostata specificando i retention periods per ogni singolo oggetto, o complessivamente per tutto il bucket.

Object Versioning

In MinIO, gli oggetti vengono versionati indipendentemente, analogamente a come viene effettuato un Amazon S3. MinIO assegna ad ogni oggetto un ID univoco ad ogni versione di ogni oggetto, le applicazioni possono specificare l'ID del file da ottenere, in modo tale da accedere in ogni momento a qualsiasi snapshot dell'oggetto.

Il versioning permette agli utenti di ottenere più variazioni di uno stesso oggetto nello stesso bucket, fornendo un meccanismo per preservare, ottenere e riportare qualsiasi versione di ogni oggetto presente all'interno di un bucket, eliminando il requisito di effettuare snapshots. Il

beneficio principale delle feature di versioning risiedono nel fatto che offrono una protezione contro cancellazioni o scritture accidentali; questo è possibile grazie al concetto del delete marker, ovvero quando un oggetto viene eliminato, non viene cancellato permanentemente, ma esclusivamente segnato come cancellato.

Data Life Cycle e Tiering

MinIO supporta funzionalità per gestire la transizione degli oggetti tramite regole di gestione del lifecycle, in modo tale da permettere a MinIO di spostare automaticamente gli oggetti in storage remoti, appartenenti ad altri tier, come:

- MinIO
- Amazon S3
- Google Cloud Storage
- Microsoft Azure Blob Storage

Le feature di object transition offerte in MinIO permettono di spostare dati vecchi da cluster MinIO (sia pubblici che privati) a soluzioni di cloud storage low cost (sia pubbliche che private).

Bucket Notifications

Le notifiche di bucket di MinIO consentono agli amministratori di inviare notifiche a servizi esterni relativamente eventi, sia a livello di oggetto che di bucket.

Per configurare le notifiche di bucket in MinIO, è necessario specificare il nome del bucket, l'evento o gli eventi per i quali si desiderano le notifiche e l'URL del target. È possibile configurare le notifiche di bucket tramite la console di MinIO o tramite l'API di MinIO.

Ecco alcuni esempi di come utilizzare le notifiche di bucket di MinIO:

- Inviare una notifica a un sistema di monitoraggio quando viene creato un nuovo oggetto in un bucket.
- Inviare una notifica a un sistema di archiviazione quando viene eliminato un oggetto da un bucket.
- Inviare una notifica a un sistema di sicurezza quando viene modificato l'accesso a un bucket.

Le notifiche di bucket di MinIO possono essere utilizzate per automatizzare le attività, migliorare la visibilità dei dati e migliorare la sicurezza.

External Identity Management

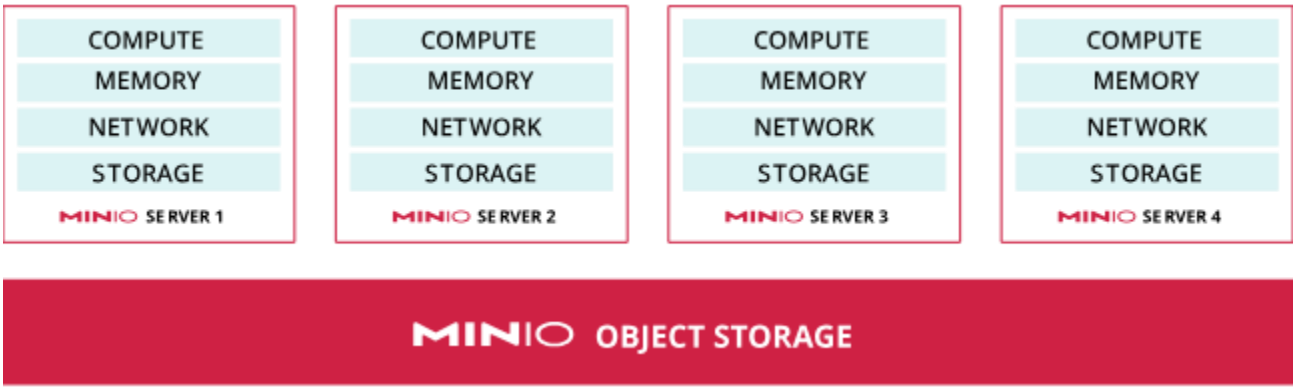
MinIO supporta diversi external identity managers come:

- OpenID compatibile (ad esempio Okta, Keycloak, Dex, Google o Facebook);
- Active Directory o LDAP.

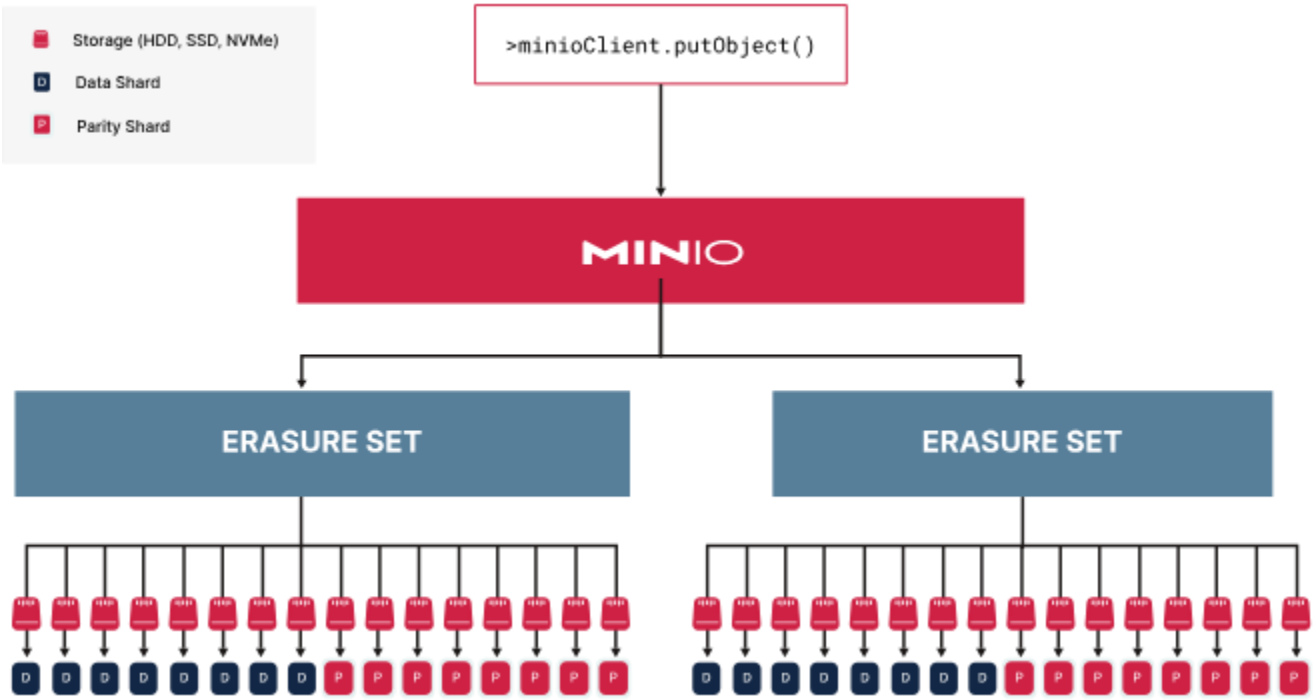
Gli utenti possono autenticarsi su MinIO utilizzando le loro credenziali utilizzate con il servizio esterno e il relativo Security Token Services per le API. Una volta autenticato, l'utente può accedere agli oggetti in base alle policies a lui assegnate.

Architettura

Un deployment tipico per la produzione comprende 4 host MinIO con risorse di storage e computazionali omogenee. MinIO raggruppa assieme tali risorse in un unico Pool in modo da presentarle come singolo servizio Object Storage:



Nell'immagine sopra, dunque, vediamo 4 host differenti che indicano un unico pool di nodi MinIO. Per sfruttare al massimo le potenzialità e le performance di MinIO è utile utilizzare loacally-attached storage come ad esempio NVMe o SSD senza alcuna configurazione RAID, pooling o altri livelli di resilienza hardwer/software. Un altro concetto importante nell'architettura di MinIO è quello di **erasure set**. Infatti, grazie ad esso, MinIO gestisce la disponibilità e la resilienza durante fallimenti di drive o nodi. Ogni oggetto viene partizionato in data e parity shards che vengono distribuiti in un unico erasure set. In altre parole MinIO automaticamente raggruppa i drives nel pool in un erasure set. Ad esempio, nell'immagine sotto, vediamo due erasure set con 16 drives e con un parity block di EC:8 ciascuno:



I data shards sono i blocchi contenenti i dati veri e propri, mentre i parity shards sono i blocchi contenenti le repliche dei data shards, utili per la ricostruzione dei data shards in casi di problemi di drive, nodi o network.

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.2 – Componente Iceberg

Table format

Il ruolo di un table format all'interno del Data Lake è quello di fornire una struttura ai dati archiviati. L'utilizzo di tale componente consente di organizzare i dati in modo coerente, rendendoli più facili da gestire e analizzare. Inoltre, grazie a questa componente, è possibile abilitare le transazioni ACID sull'Object Storage.

I table format possono essere utilizzati per diversi scopi, tra cui:

- Descrivere la struttura dei dati: Indicando il tipo di dati di ciascuna colonna e la sua lunghezza. Questo consente di capire come sono formati i dati e di utilizzarli in modo corretto.
- Effettuare query sui dati: Molti strumenti di analisi supportano l'utilizzo di table format. Questo consente di eseguire query sui dati in modo efficiente e preciso.
- Elaborare i dati: I table format possono essere utilizzati per elaborare i dati in modo efficiente. Ad esempio, possono essere trasformati o aggregati prima di essere analizzati.

La scelta del table format più adatto dipende da diversi fattori, tra cui il tipo di dati da archiviare, le esigenze di analisi e le prestazioni richieste. In particolare, all'interno di un Data Lake, l'utilizzo di un table format può apportare i seguenti benefici:

- Migliora la leggibilità e la comprensibilità dei dati: Il table format fornisce una descrizione chiara della struttura dei dati, rendendoli più facili da leggere e comprendere.
- Facilita l'integrazione dei dati da diverse fonti: I table format possono essere utilizzati per integrare i dati da diverse fonti, anche se queste fonti utilizzano formati diversi.
- Migliora le prestazioni delle query: L'utilizzo di un table format efficiente può migliorare le prestazioni delle query, rendendole più veloci ed efficaci.

Apache Iceberg

Apache Iceberg è un Table Format open source molto performante, che tramite query SQL permette l'analisi di grandi quantità di dati. Inoltre, offre le seguenti feature:

- Supporta lo Schema Evolution di una tabella consentendo di aggiungere, eliminare, aggiornare e rinominare le colonne di una tabella senza effetti indesiderati.

- Hidden Partitioning che consente di ridurre al minimo gli errori ed evitare di avere query molto lente.
- Time Travel che consente di effettuare le query su versioni diverse della medesima tabella in base al timestamp oppure alla versione.
- Version Rollback che consente di tornare ad uno stato antecedente qualora sorgano dei problemi con una tabella.

Queste feature permettono una più facile gestione dei dati all'interno del Data Lake, non preoccupandosi più come prima dei cambiamenti effettuati sui dati in produzione. Iceberg permette a più engine in contemporanea di operare su uno stesso dataset. Abbiamo scelto di utilizzare questo Table Format in quanto è supportato in Dremio per la lettura e la scrittura dei metadati su sorgenti di tipo S3, come ad esempio MinIO. Con Dremio è possibile effettuare le funzioni di Data Manipulation Language come INSERT, UPDATE, DELETE e MERGE INTO oltre che le classiche funzioni per la lettura dei dati. Tramite file di metadati, Iceberg tiene traccia degli snapshot nel tempo mantenendo tutti i delta come tabella. Ogni snapshot fornisce una descrizione completa delle informazioni sullo schema, sulla partizione e sul file della tabella. Inoltre, Iceberg organizza in modo intelligente i metadati degli snapshot in una struttura gerarchica. Ciò consente a Dremio di apportare modifiche rapide ed efficienti alle tabelle senza ridefinire tutti i file del set di dati, garantendo così prestazioni ottimali quando si lavora su scala di Data Lake.

5.3 – Componente Project Nessie

Table Repository

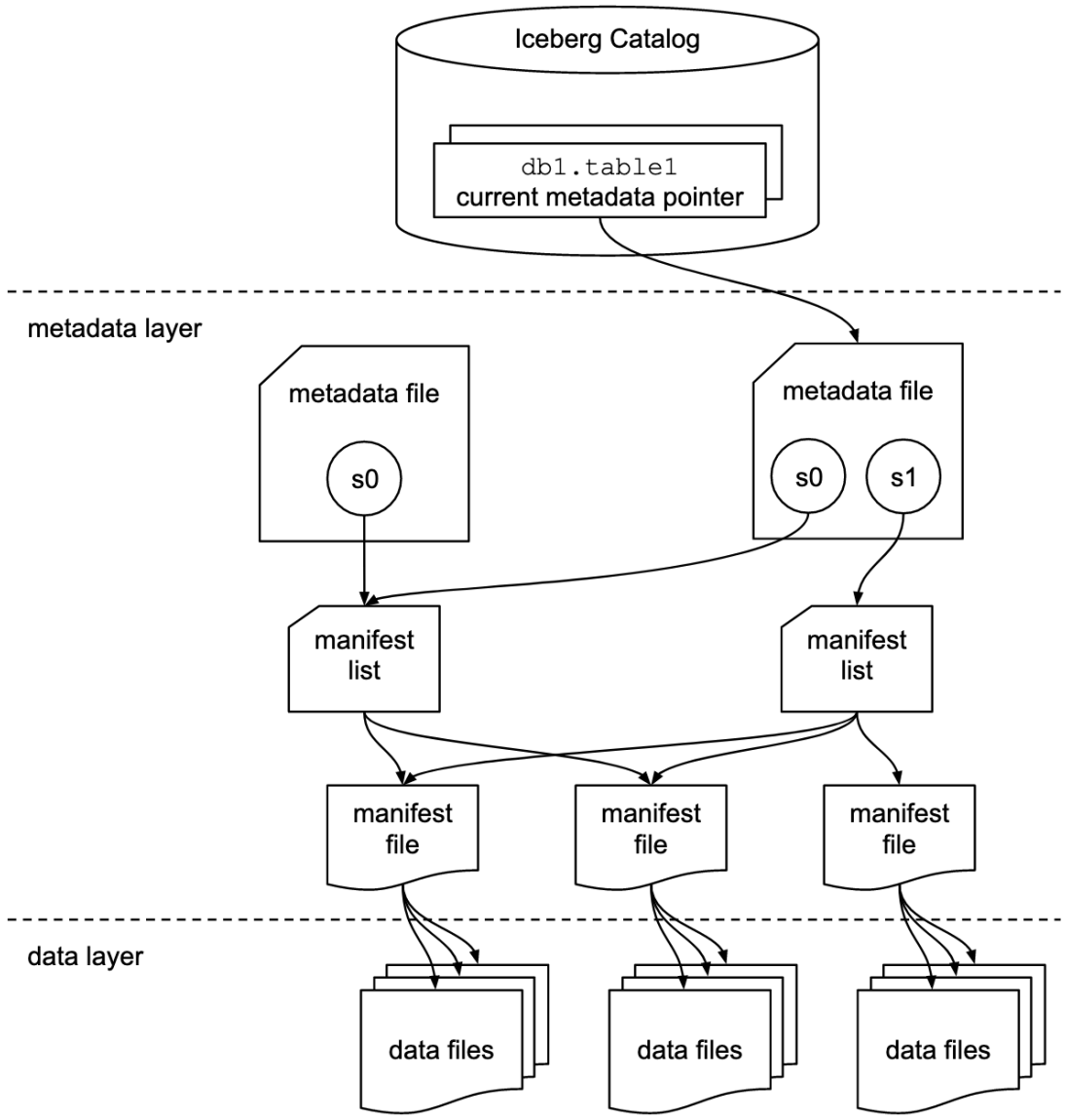
Il funzionamento di Iceberg è supportato dai file dei metadati che fanno riferimento ai dati veri e propri. Quindi, è necessario fornire ad Iceberg il giusto puntamento ai metadati, ed è proprio questo il ruolo del Table Repository. Ci sono diversi tipi di Table Repository, come Hive Metastore, AWS Glue oppure Project Nessie. Sarà proprio quest'ultimo che utilizzeremo come Table Repository, motivando questa scelta nel paragrafo successivo.

Project Nessie

Project Nessie è un catalogo transazionale open source utilizzato nei Data Lake. Oltre ad assumere il ruolo di Table Repository per le tabelle Iceberg, è molto utile perchè abilita le funzionalità Git-like nel Data Lake. Per questo motivo abbiamo individuato questa tecnologia come Table Repository, dato che non si limita al solo ruolo di catalogo transazionale. È possibile abilitare il versionamento non solo su una tabella alla volta come permetteva Iceberg, ma anche su più tabelle contemporaneamente. Consente, dunque, la creazione di branch sui quali apportare modifiche, mantenendo consistenti i dati e permettendo di fare un merge quando

siamo certi che le modifiche effettuate siano corrette. Project Nessie, quando gestisce più versioni di una tabella in diversi branch, non applica una copia dei dati ma si limita a gestire i metadata file delle tabelle Iceberg. Abbiamo scelto di proporre questa tecnologia in quanto è supportata anche da Dremio dato che supporta, tra gli altri, anche Project Nessie come Metadata Catalog per Apache Iceberg. Oltre a questo, come storage supporta Object Storatge di tipo S3 come, ad esempio, MinIO.

Per mettere a fattor comune quanto illustrato fino ad ora, illustriamo il tutto con una immagine:



In questa architettura possiamo osservare la gestione dei file attraverso i metadati effettuata da Apache Iceberg. Inoltre, quello che è denominato come Iceberg Catalog è a tutti gli effetti Project Nessie: il Table Repository che tiene traccia dell'attuale puntatore ai metadati. Il metadata layer è composto da tre componenti:

- Metadata file: include informazioni sullo schema della tabella, informazioni sulle partizioni, sugli snapshot e circa lo snapshot attuale.
- Manifest list: contiene una lista di manifest file assieme alle informazioni dei manifest file che hanno effettuato uno snapshot.
- Manifest file: tracciano i data file oltre che avere altri dettagli e statistiche per ogni file

Il data layer contiene i data files veri e propri. Tutta questa struttura illustrata fino ad ora, dall'Icebrg Catalog fino al Data Layer può risiedere su un Object Storage di tipo S3, come MinIO. Inoltre può interagire senza problemi con Dremio, abilitando di fatto a quello che è a tutti gli effetti un Data Lakehouse.

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.4 – Componente Dremio

Layer di Data Access

Il Data Access funge da ponte tra gli utenti e le sorgenti di dati presenti nell'infrastruttura.

La sua funzione principale è quella di fornire un'interfaccia unificata per l'accesso ai dati, spesso utilizzando linguaggi standard come SQL. Questo permette agli utenti di eseguire query e analisi sui dati senza la necessità di conoscere i dettagli di implementazione delle varie sorgenti dati.

Inoltre, un data access gestisce le autorizzazioni degli utenti, garantendo l'accesso ai dati sensibili solo a coloro che hanno i permessi necessari senza dover conoscere le specificità tecniche come, ad esempio la tecnologia utilizzata, il formato dei dati o la modalità di accesso a ciascuna sorgente.

Questo componente può anche includere funzionalità di sicurezza avanzate per proteggere i dati durante l'accesso e la trasmissione.

All'interno dell'architettura proposta, il Data Access sarà la modalità di accesso standard ai dati presenti nell'architettura.

Dremio

Dremio nella nostra infrastruttura, implementa lo strato di data access. E' in grado di fornire una modalità unificata per l'interrogazione e la modellazione dei dati presenti nell'architettura, tramite linguaggio SQL.

La funzionalità principale di Dremio è quella di Data Virtualization, in quanto permette di astrarre sorgenti dati diverse come se fossero un'unica sorgente. Questo consente agli utenti di effettuare operazioni di join tra sorgenti eterogenee senza bisogno di convertire i formati di partenza o integrare le diverse sorgenti.

Oltre alle feature appena descritte, Dremio offre possibilità di integrazione con diversi tool di Business Intelligence come, ad esempio, Power BI, Superset, Tableau e così via.

Caratteristiche principali

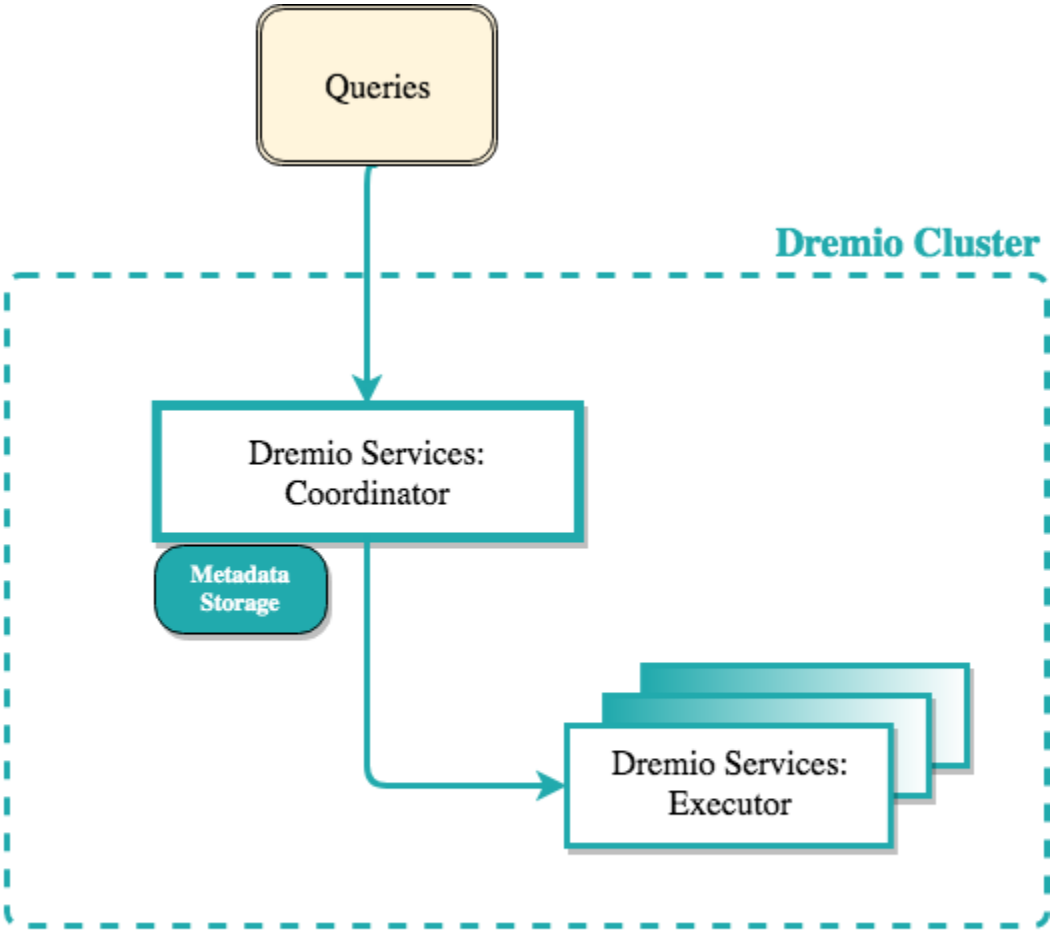
Tra le astrazioni principali di Dremio, abbiamo:

- Sources: Si tratta delle sorgenti presenti all'interno di Dremio; si dividono in: Metastore, Nessie Catalogs, Object Storage, Databases o Files and Directories. Rappresentano la modalità con cui ottenere dati, e gestirli all'interno di Dremio.
- PDS (Physical DataSet): L'oggetto che rappresenta il dato raw presente all'interno del database.
- VDS (Virtual DataSet): Una tabella virtuale creata da una o più VDS o PDS.
- Reflection: Consiste nel salvataggio in cache di una VDS o PDS, al fine di velocizzare notevolmente l'esecuzione di query molto dispendiose.
- Semantic Layer: Consiste nella divisione logica delle VDS a seconda del loro stato di aggregazione. Partiamo dal livello Staging in cui sono presenti le SELECT * delle PDS, fino ad arrivare al livello Application in cui sono presenti le VDS con i risultati pronti per gli strumenti di BI. Questa suddivisione logica è utile anche per la segregazione degli utenti.
- Spaces: Cartelle all'interno delle quali vengono salvate le VDS virtualizzate all'interno di Dremio. Rappresentano la funzionalita' con cui raggruppare i dataset secondo varie modalità, quali dipartimento, progetto, scopo, e così via.
- Security: Con la versione Enterprise è possibile segregare gli utenti in base al ruolo, concedendo i giusti privilegi sugli Space, Cartelle o VDS/PDS.
- Data Catalog: è possibile ricercare le VDS o le PDS presenti in Dremio per nome. Inoltre, ogni VDS ha associato un Wiki che può essere compilato descrivendone l'utilizzo. È inoltre possibile inserire tag e visualizzare lo schema delle VDS.
- Data Lineage: Dremio fornisce la lineage di una VDS così da conoscerne la sua composizione dalla sorgente fino alla fine.

Architetturalmente Dremio comprende due diversi tipi di nodi:

- Coordinator: è il nodo che gestisce le query effettuate dagli utenti sia da interfaccia che tramite driver ODBC, JDBC o Arrow Flight, ne calcola il costo e le distribuisce agli executor. Inoltre, è il nodo che espone l'interfaccia web.
- Executor: è il nodo che prende in carico la query dal coordinator e la esegue.

Quindi, un'architettura di massima di Dremio potrebbe essere la seguente:


Queries
Dremio Cluster
Dremio Services:
Coordinator
Metadata
Storage
Dremio Services:
Executor

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.5 Componente Orchestrator

Mage

Mage è un tool open source e free to use che permette lo sviluppo e l'orchestrazione di pipeline di trasformazione e integrazione di dati. In quanto orchestratore e gestore di flussi ELT, questo tool potrebbe essere utilizzato per sviluppare le pipeline da eseguire all'interno dell'architettura e per monitorare l'esecuzione di tali flussi, in modo tale da avere traccia delle varie esecuzioni avvenute e salvare i log di tali esecuzioni.

Tra le funzionalità principali, abbiamo:

1. Gestire l'integrazione e la sincronizzazione di sorgenti e destinazioni dati di terze parti
2. Costruire pipeline streaming o batch, utilizzando Python, R o SQL
3. Eseguire, monitorare ed orchestrare l'esecuzione di pipeline ELT

Concetti chiave

In Mage, sono definiti i seguenti concetti chiave:

- Progetto: associabile ad una repository Git, intesa come il "luogo" dove viene scritto il codice di esecuzione delle pipeline.
- Pipeline: contiene le relazioni tra i vari blocchi (BLocks) di codice che devono essere eseguiti; serve ad organizzare la dipendenza ed il flusso di esecuzione dei vari blocchi
- Block: un file di codice, potenzialmente eseguibile indipendentemente o all'interno di una pipeline.
- Data Product: i file/logs/dati che vengono prodotti a seguito dell'esecuzione di ogni Block.
- Trigger: un set di istruzioni che determinano il momento quando e come deve essere eseguita una pipeline.
- Run: contiene informazioni quali momento di inizio, status, momento di fine e ogni tipologia di variabili di runtime, relativamente all'esecuzione di un Block o Pipeline.

Autenticazione

Mage supporta la creazione, gestione e l'autenticazione delle utenze. Tramite l'interfaccia grafica degli utenti aventi il ruolo OWNER, è possibile gestire la creazione, modifica, reset della password e la cancellazione degli utenti. Come modalità di autenticazione, viene fornita la possibilità di gestire gli accessi tramite protocollo LDAP. Una volta autenticati tramite questo

metodo, Mage attribuirà ad ogni utente i permessi a lui associati, basandosi all'appartenenza ai gruppi LDAP a cui l'utente appartiene. Come altri metodi di autenticazione, è possibile utilizzare Google, Okta o Microsoft.

Nel caso in cui venga attivata l'autenticazione, ogni richiesta API effettuata al server Mage, dovrà essere autenticata tramite un API key ed un token OAuth2.

Role-based Access Control

Al fine di gestire le operazioni e gli spazi a cui un utente ha accesso, Mage implementa Roles e Pemissions. A livello logico, vengono definiti permessi, i quali vengono assegnati a ruoli che verranno poi attribuiti agli utenti.

Permissions sono la modalità per garantire o negare l'accesso in scrittura e/o lettura a specifiche entità (pipelines o blocks), attribuendo permessi di richiamare specifici API Endpoint.

I Roles sono la modalità con cui attribuire Permissions ai singoli utenti, nel dettaglio:

- Un ruolo può essere composto da 1 o più permessi
- Un utente può avere uno o più ruoli
- Ogni ruolo deve essere identificato tramite un nome univoco

Divisione per workspace

Mage permette di dividere le aree di lavoro in workspace, una volta abilitata questa divisione, sarà possibile assegnare permessi agli utenti a livello di workspace. Al fine di effettuare questa tipologia di deployment, è necessario effettuare un deploy della tecnologia in modalità cloud-native, ad esempio tramite Kubernetes (sviluppi futuri garantiranno l'integrazione anche con Amazon ECS e Google Cloud Run).

UI

Tramite l'Interfaccia Utente di Mage, è possibile visualizzare e modificare sia le pipeline che le impostazioni del cluster, come ad esempio, gestire ruoli, permessi ed utenti, le impostazioni per l'esecuzione delle pipeline, e così via.

Pipelines

Per quanto riguarda la possibilità di modificare le pipeline, Mage offre un'interfaccia "Notebook-based", la quale permette di:

Visualizzare le pipeline esistenti e monitorare le loro esecuzioni

Modificare singolarmente ogni task di una pipeline, scrivendo codice Python, R o SQL direttamente all'interno dell'interfaccia Mage

Modificare il comportamento generale della pipeline, spostando la catena di esecuzione delle task in modalità interattiva

Definire la modalità di triggering delle pipeline

Gestione delle dipendenze

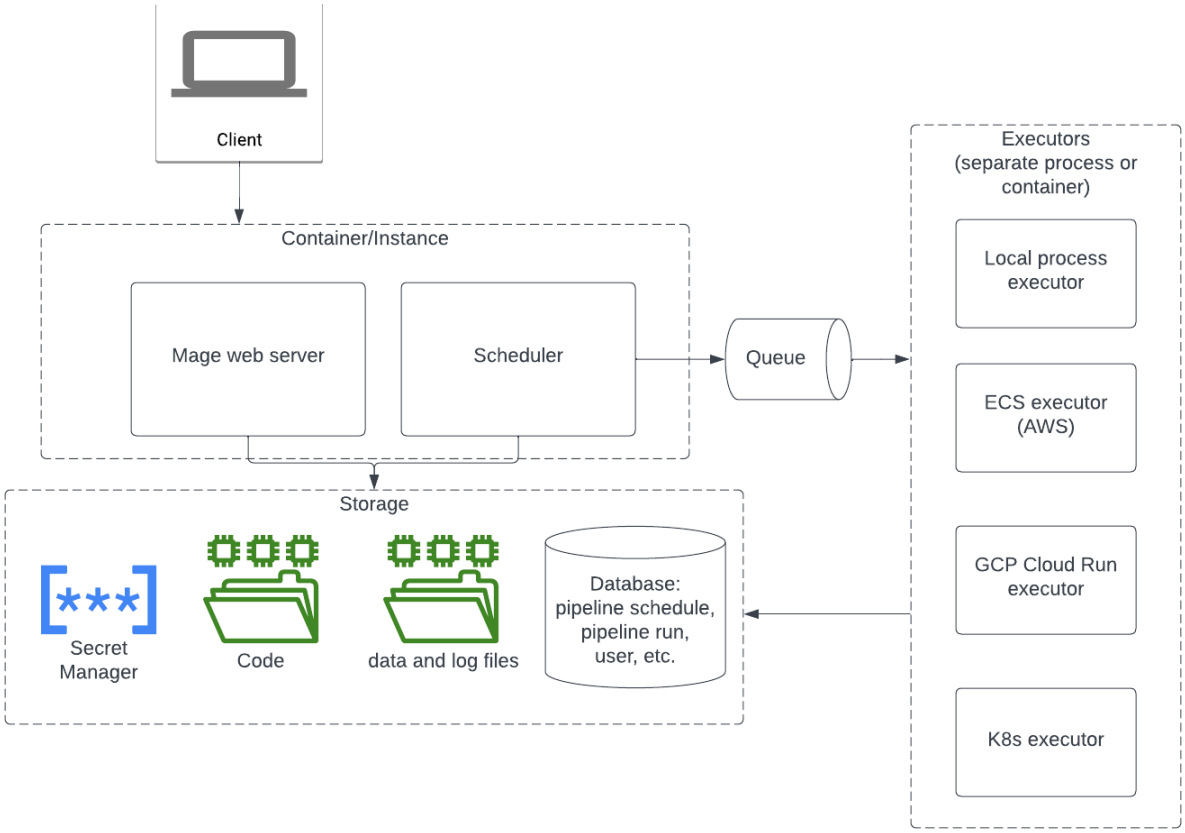
Mage offre la possibilità di definire catene di dipendenze extra-flusso, ovvero di specificare un vincolo di dipendenza tra due pipeline distinte.

Modalità di triggering delle pipeline

Per lanciare l'esecuzione di una pipeline, Mage offre diverse modalità di triggering:

- Schedule: basato su informazioni temporali
- Event: basato sull'occorrenza di eventi
- API: la pipeline viene triggerata quando viene effettuata una specifica chiamata api

A seguire un diagramma dell'architettura di Mage:



Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

Apache Airflow

Airflow è una piattaforma open-source che supporta lo sviluppo, schedulazione e monitoraggio di workflow batch-oriented. Tramite l'utilizzo del framework Python per Airflow, è possibile costruire flussi per ogni tecnologia. Tramite l'utilizzo dell'interfaccia web. è possibile gestire lo stato dei flussi integrati all'interno dell'istanza in esecuzione.

Apache Airflow non è basato su interfacce a linea di comando o XML. Al fine sviluppare i flussi ELT in Airflow, viene fornito un framework Python, il quale permette di creare workflows, gestire la schedulazione di questi ultimi, e modalità per la generazione automatica di nuove task.

Concetti chiave

I concetti chiave presenti in Apache Airflow sono:

- DAGs (Dyrected Acyclic Graph): concetto principale in Airflow, caratterizzato da una collezione di Tasks, organizzate tramite dipendenze e relazioni, che definiscono la modalità di esecuzione.
- DAG Runs: si tratta di un oggetto rappresentante l'istanza di un DAG nel tempo; ogni volta che un DAG viene eseguito, viene creato un DAG Run, nel quale vengono eseguite tutte le task di una DAG.
- Tasks: l'unità di esecuzione più basica in Airflow, i quali vengono raggruppati in DAGs; possono avere dipendenze upstream e downstream, in modo da definire l'ordine secondo cui devono essere eseguite.
- Operators: si tratta concettualmente di un template per una task predefinita, dichiarabile all'interno di un DAG
- Sensors: tipologia di Operators specializzati ad eseguire una task definita, come l'attesa di un evento.
- Executor: meccanismo secondo il quale vengono eseguite le Task instances.

Autenticazione

Tramite Airflow è possibile definire politiche di autenticazione.

Role-based Access Control

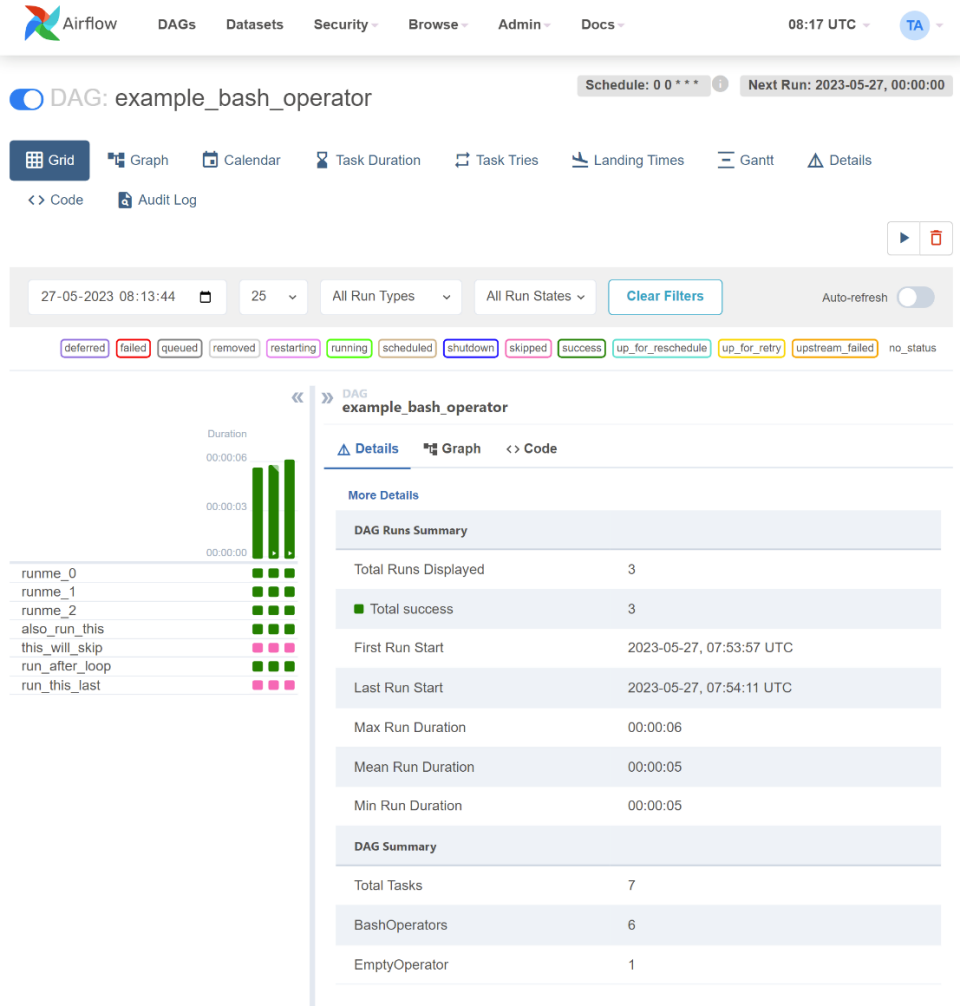
In Airflow viene fornita la possibilità di attribuire ruoli agli utenti in modo tale da definire sia le azioni che ogni utenza può effettuare all'interno dell'ambiente, che definire chi può vedere un determinato set di DAG.

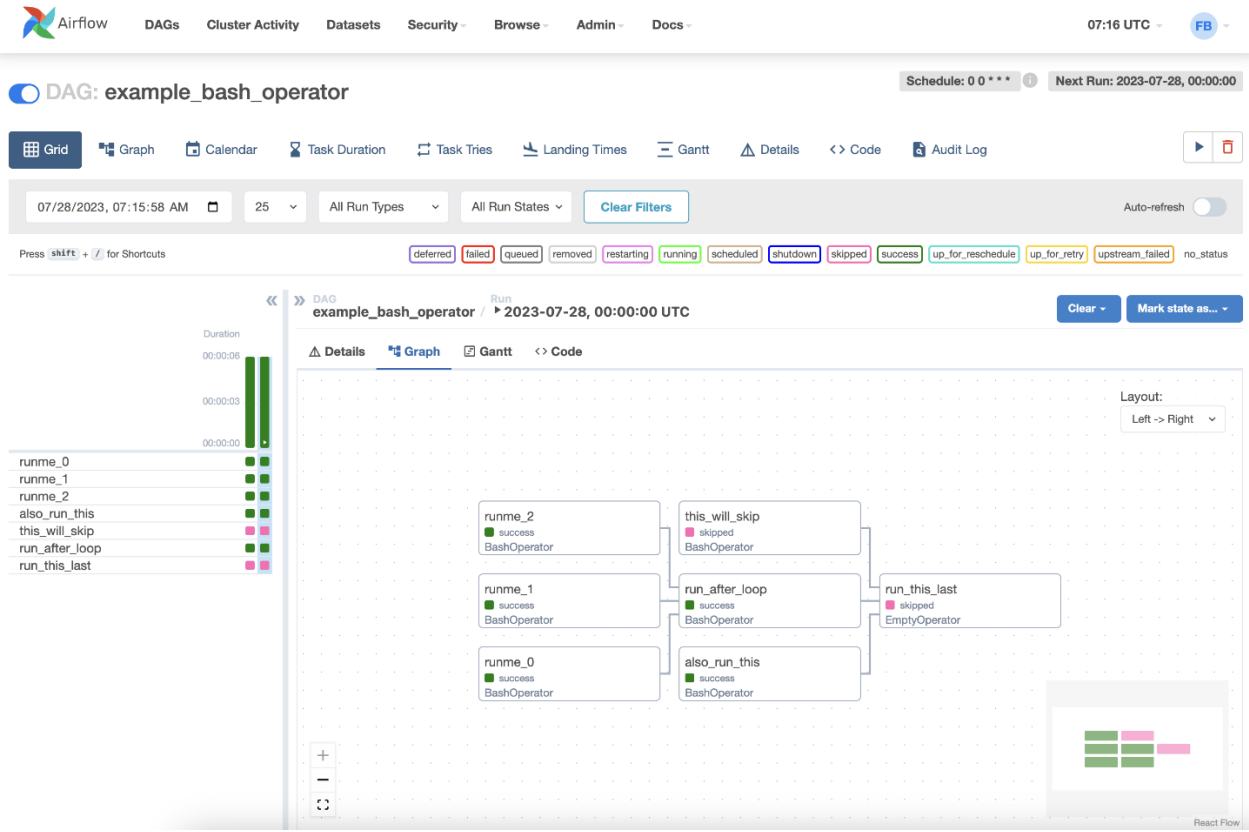
UI

All'interno di Airflow, l'interfaccia utente consente di visualizzare le task presenti, lanciare run, controllare LOGs ed effettuare delle basilari operazioni di debug.

Monitoraggio dei DAG

Tramite l'interfaccia grafica di Airflow, è possibile monitorare l'esecuzione dei dag, come si può vedere nei seguenti screenshot.

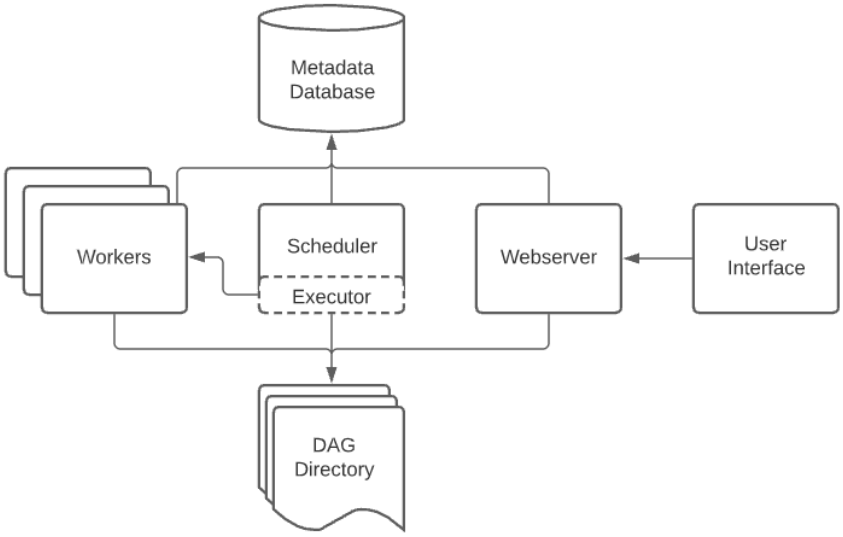




Gestione delle dipendenze

Airflow consente la possibilità di definire catene di dipendenza tra DAG, operazione possibile tramite l'ExternalTaskSensor, integrato in Airflow.

A seguire un diagramma che ritrae l’architettura di AirFlow:


Metadata Database
Workers
Scheduler
Executor
Webserver
User Interface
DAG Directory

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

Nifi

Nifi è un gestore di flussi dati, supporta lo sviluppo di grafi di instradamento e trasformazione di dati. Tramite l'UI, è possibile sviluppare, monitorare e controllare i dataflow. è possibile configurare il servizio in base a differenti dimensioni di qualità di servizio, come, ad esempio, bassa latenza, arrivo garantito dei dati, e così via.

Nifi è un applicativo molto maturo e importante all'interno della community Apache. La prima release risale al 2006, da allora è stato manutenuto e aggiornato a molteplici versioni. Ora, Nifi è un software incredibilmente affidabile nella gestione e orchestrazione di flussi ELT.

Concetti chiave

I concetti principali all'interno di Nifi sono:

- FlowFile: rappresenta ogni oggetto in movimento tra sistemi.
- FlowFile Processor: il componente che esegue le computazioni, nel dettaglio, vengono eseguite le operazioni di data routing, trasformazioni e/o mediazioni tra sistemi.
- Connection: fornisce una modalità di collegamento tra processors. Questi funzionano come code, permettendo i vari processori di interagire.
- Flow Controller: funziona da broker, in modo da agevolare lo scambio di FlowFiles, tra i vari processors.
- Process Group: insieme di processi e connettori i quali possono ricevere dati da porte di input ed inviare dati alle porte di output.

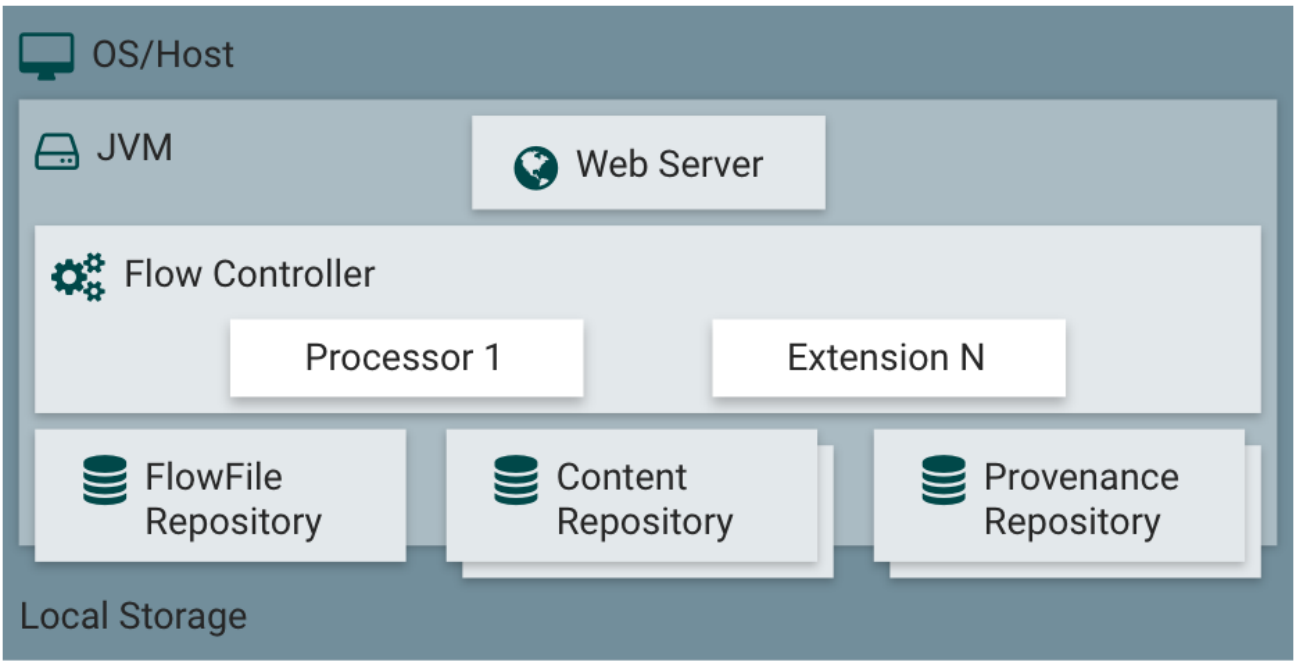
Autenticazione

In Nifi, l'autenticazione tramite Username e Password è supportata tramite un "Login Identity Provider"; viene inoltre offerta la possibilità di autenticarsi tramite: Single User, Lightweiht Directory Access Protocol (LDAP) e Kerberos.

Role-based Access Control

Nifi supporta la gestione dei ruoli, offrendo agli amministratori la possibilità di impostare sia le modalità che le aree in cui i vari utenti possono accedere.

Architettura Nifi



Nifi viene seguito all'interno di una JVM, i suoi componenti principali sono:

- Web Server: Effettua l'host dei comandi HTTP e delle API di Nifi.
- Flow Controller: Fornisce threads per permettere l'esecuzione di Extensions e Processors.
- FlowFile Repository: L'area di memoria dove vengono salvati gli stati dell'esecuzione dei vari flussi.
- Content Repository: Dove vengono salvati i contenuti di un FlowFile.
- Provenance Repository: La locazione dove vengono salvati i dati di Provenance.

Divisione per workspace

Tramite funzionalità di Multi-Tenancy, si ha la possibilità di definire gruppi di utenti (tenants), i quali possono accedere a differenti aree di lavoro, con diversi livelli di autotizzazione. Le policy di accesso alle varie aree del software possono essere definite sia in modalità "system wide" che singolarmente per ogni singolo componente dell'architettura.

UI

L'interfaccia grafica di Nifi permette di creare, visualizzare, modificare, monitorare e amministrare i dataflows. Tramite la UI, è inoltre possibile gestire le informazioni relativi gli utenti presenti nella piattaforma e le relativa policy di accesso.

Gestione delle dipendenze

Per definire dipendenze tra flussi Nifi, è possibile definire tutti gli step all'interno di una stessa pipeline, oppure, tramite l'utilizzo dei pattern wait/notify, è possibile attendere l'esecuzione di un'altra pipeline, prima di avviare l'esecuzione di un flusso.

Proposta di dispiegamento

Per quanto riguarda l'implementazione delle funzionalità di orchestrazione e gestione di flussi ELT all'interno dell'architettura proposta, abbiamo definito tre possibili scenari:

1. Airflow e Nifi, in questa proposizione di architettura, verrebbero utilizzati sia Airflow che Nifi per l'orchestrazione e lo sviluppo di flussi ELT.
2. Mage, utilizzare questo tool come unico strumento di orchestrazione e sviluppo/gestione dei flussi ELT. Essendo un tool molto giovane, bisogneranno effettuare dei test di carico, al fine di vedere le sue capacità a livello di processamento delle informazioni.
3. Sostituire Airflow o Nifi con Mage, per eseguire sia l'orchestrazione che lo sviluppo dei flussi ELT.

Orchestratore

L'orchestratore, all'interno di un'infrastruttura dati, è responsabile dell'esecuzione dei flussi ELT (Extract, Transform, Load) e del monitoraggio del loro funzionamento.

Le funzioni principali dell'orchestratore comprendono:

1. Programmazione e Avvio: L'orchestratore si occupa della programmazione e dell'attivazione dei flussi ELT secondo specifici trigger temporali, eventi o condizioni.
2. Monitoraggio Dettagliato: Fornisce strumenti per il monitoraggio approfondito dei flussi ELT in esecuzione, presentando informazioni chiare sullo stato delle attività, sui tempi di esecuzione e su eventuali incongruenze o errori.
3. Supporto allo Sviluppo: Molti strumenti di orchestrazione includono feature per la visualizzazione del flusso dati, la gestione delle dipendenze, la fase di testing e di sviluppo.

4. Gestione delle Autorizzazioni Utente: L'orchestratore include funzionalità per la gestione delle autorizzazioni degli utenti, determinando i privilegi di accesso e le modalità di interazione con le diverse aree del software dedicate all'elaborazione dati. Questo aspetto comprende la definizione di ruoli e permessi, specificando chi può accedere a determinati flussi ELT, quali azioni sono permesse (ad esempio, lettura, scrittura, modifica) e in che misura.

In conclusione, l'orchestratore svolge un ruolo determinante nell'esecuzione, monitoraggio e sviluppo dei flussi ELT all'interno dell'infrastruttura dati. La sua presenza garantisce efficienza e tempestività nell'elaborazione dei dati, fondamentali per l'ambito progettuale e operativo dell'architettura dati.

Gestore flussi ELT

Il gestore di flussi ELT è progettato per facilitare la creazione, lo sviluppo e la gestione delle pipeline ELT. Questo tipo di software offre una serie di funzionalità specifiche:

1. Connettività alle Fonti Dati: Fornisce connettori predefiniti o strumenti per connettersi a diverse fonti di dati come database, file, servizi cloud, API e altre sorgenti, semplificando l'estrazione dei dati da diverse origini.
2. Strumenti di Trasformazione: Include una varietà di strumenti e funzioni predefinite per la trasformazione dei dati durante il processo ELT. Questi strumenti consentono di manipolare, pulire, normalizzare e trasformare i dati per adattarli al formato richiesto dal sistema di destinazione.
3. Programmabilità e Scripting: Permette agli sviluppatori di scrivere script o codice per eseguire trasformazioni all'interno delle pipeline ELT.
4. Gestione delle Dipendenze e del Controllo di Flusso: Consente di gestire le dipendenze tra le diverse fasi della pipeline e di definire il controllo del flusso dei dati, stabilendo l'ordine e le condizioni per l'esecuzione delle operazioni.

In generale, un software gestore di flussi ELT offre strumenti completi e personalizzabili per creare, gestire e ottimizzare le pipeline ELT, semplificando il processo di movimento e trasformazione dei dati all'interno di un'infrastruttura dati complessa.

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.6 – Componente Kafka

Apache Kafka

Apache Kafka è un sistema di streaming distribuito open-source che si occupa di Stream Processing, Real-time data pipeline e data integration su larga scala. Kafka è ampiamente utilizzato ed è diventato presto uno standard de facto per lo sviluppo di applicazioni di nuova generazione che come obiettivi si prefiggono la scalabilità e il data streaming real-time. Le caratteristiche che hanno permesso la sua affermazione sono:

- Alto throughput: Kafka riesce a gestire ampi volumi di dati ad un'alta velocità;
- Alta scalabilità: il cluster Kafka può essere facilmente ampliato o ridotto a seconda dei carichi di messaggi che deve gestire;
- Bassa latenza: riesce a spedire i messaggi arrivati con una latenza nell'ordine dei millisecondi;
- Storage permanenti: i dati ricevuti possono essere salvati in maniera durabile, affidabile e distribuita, garantendo la fault-tolerance;
- Altamente disponibile: è possibile estendere il cluster Kafka in altre zone di disponibilità in modo da essere fault tolerant in caso di guasti o anomalie.

Per garantire le caratteristiche descritte sopra, Apache Kafka adotta un funzionamento Publish/Subscribe: i Producer sono le entità che scrivono su Kafka pubblicando i messaggi, mentre i Consumer sono le entità che leggono i messaggi da Kafka sottoscrivendosi ad un determinato topic. Possono esserci più producer che scrivono su un topic e più consumer che leggono da un topic. Andiamo a descrivere alcuni dei tool più importanti presenti in Kafka:

- Kafka Connect: permettono il dialogo tra Kafka e le sorgenti/destinazioni esterne, hanno un basso overhead garantendo alte performance. La suite di Confluent Kafka, come detto in precedenza, amplia la gamma di connettori disponibili. Ci sono due tipologie di Connect:
    - Source connector: permette l'ingestion di interi database e di aggiornamenti delle stream table verso un topic Kafka;
    - Sink connector: permette l'invio di dati da un topic Kafka verso indici secondari come ad esempio Elasticsearch.
- Kafka Stream: libreria che permette lo sviluppo di applicazioni in grado di prendere in input i dati provenienti da un topic e restituirli modificati e arricchiti a seconda degli scopi dell'utente. Permette quindi, tra le altre, operazioni di filtraggio, raggruppamento, aggregazione e joining.

Apache Kafka è un servizio Java based, per cui come pre-requisito necessario vi è quello di avere una versione Java pari o superiore a Java 8.

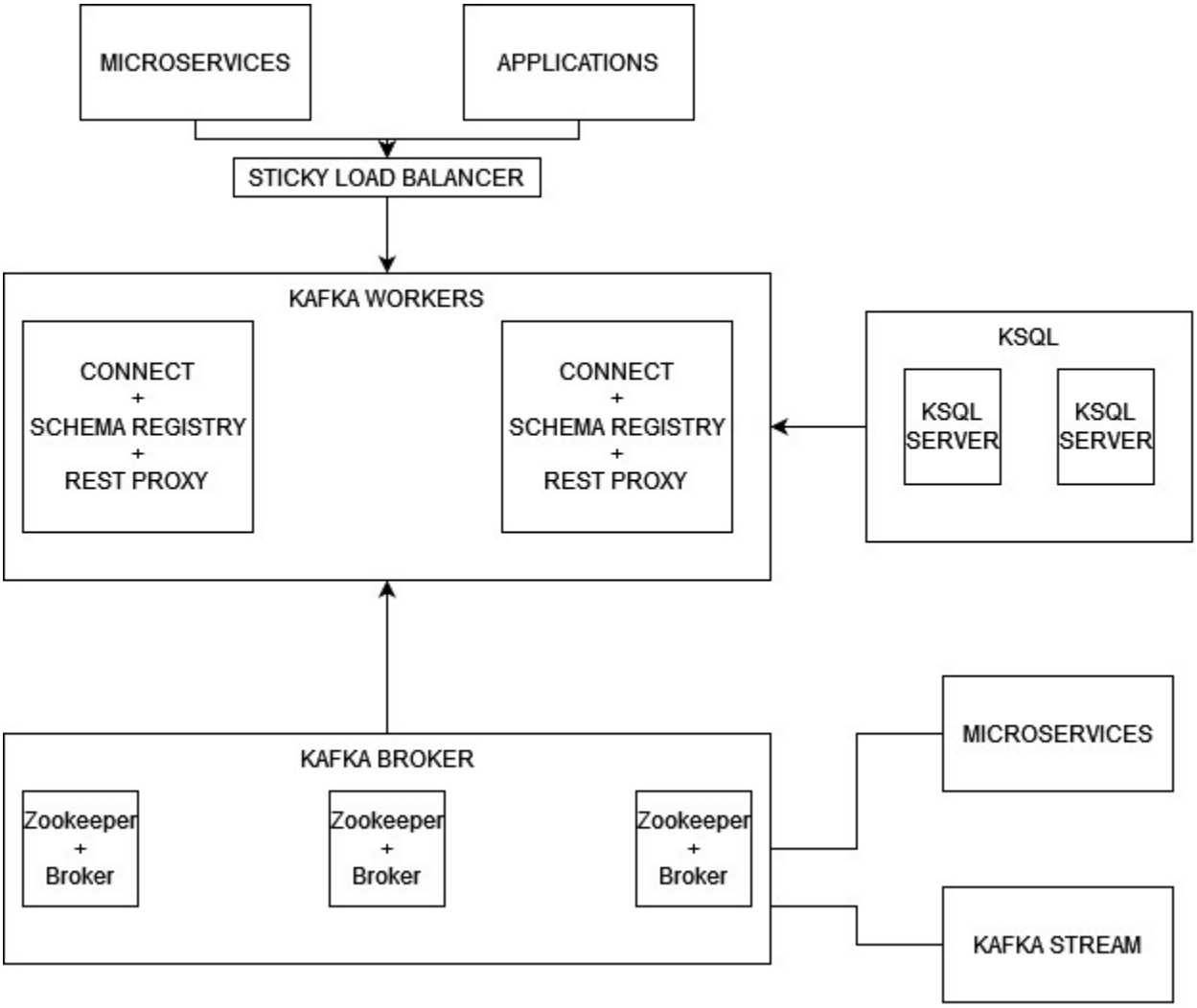
Confluent Kafka

Come detto in precedenza, essendo uno standard de facto nei suoi ambiti di utilizzo, Kafka può interagire facilmente con diverse tecnologie e questo è elevato alla massima potenza nella suite Confluent Kafka. Infatti, Confluent migliora Apache Kafka aggiungendo funzionalità community e commerciali per incrementare l'esperienza dello streaming processing andando ad aumentare ad esempio la gamma dei Connect disponibili o strumenti per la gestione del cluster. Tra questi, sicuramente i più importanti sono:

- KsqlDB: disponibile nella suite Confluent Kafka, è un database che facilita la gestione dei Kafka Stream, materializzandoli e accedendoli in maniera SQL-like anzichè prevedendo lo sviluppo di applicazioni.
- Kafka Rest Proxy: questa componente espone un endpoint REST con il quale poter comunicare con Kafka.
- Schema Registry: processo esterno che non necessariamente risiede nelle stesse macchine dei Kafka Broker. Mantiene un database di tutti gli schemi che sono stati scritti nei vari topic del cluster di cui è responsabile. Il suddetto database è persistente internamente in un topic Kafka ed è salvato in cache nello Schema Registry per un accesso a bassa latenza. Può avere una configurazione ridondante per garantire alta disponibilità. Lo Schema Registry espone anche le API che permette a produttori e consumatori di sapere se il messaggio che devono produrre o consumare è compatibile con una versione precedente o meno, sapendo quindi in che modo serializzare o deserializzare quel messaggio.

Architettura

Una architettura di massima con tutte le componenti Confluent Kafka in gioco è la seguente:



Possiamo notare che nel cluster dei broker Kafka sono presenti i Broker stessi con gli Zookeeper. Le versioni più recenti stanno andando verso la sostituzione dello Zookeeper a favore della tecnologia KRaft. Gli applicativi e le Kafka Stream possono comunicare direttamente con il cluster Kafka. Anche i Worker Node, che ospitano lo Schema Registry, il Kafka Rest Proxy e i Kafka Connect comunicano con i Kafka Broker. I worker node comunicano anche con gli applicativi e microservizi esterni (si pensi ad un Connect che movimenta dati da Kafka verso MinIO ad esempio). Infine il cluster Ksql è collegato direttamente con i Kafka Broker.

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

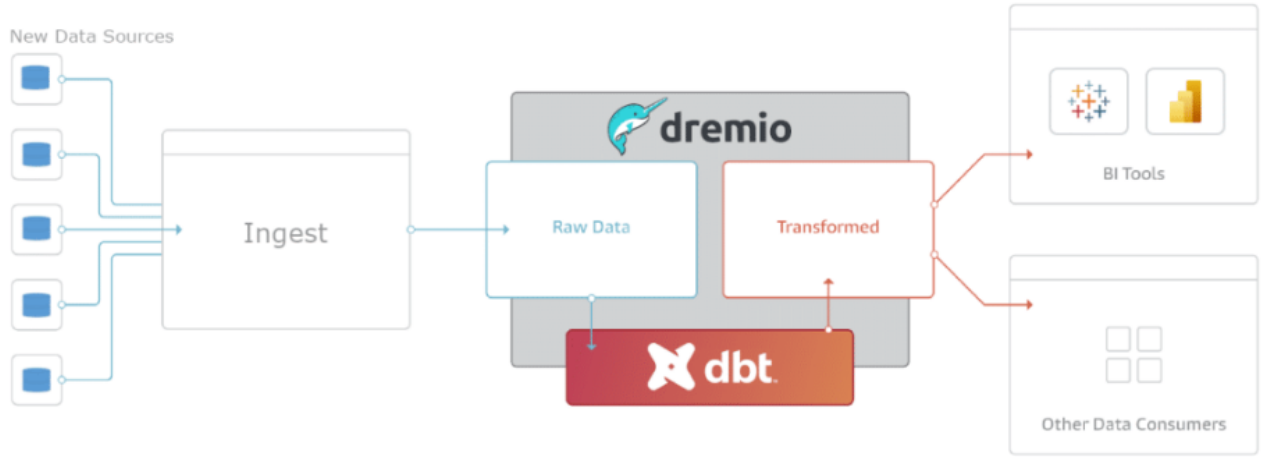
5.7 – Componente dbt

DBT (Data Build Tool)

DBT è uno strumento per la trasformazione e la preparazione dei dati all'interno dell'infrastruttura dati. Tra le funzionalità principali di DBT abbiamo:

1. Trasformazione dei dati: DBT è utilizzato principalmente per trasformare dati grezzi in un formato strutturato, pronto per l'analisi.
2. Version control: Simile ai sistemi di controllo versione del codice come Git, DBT consente il versionamento del codice di trasformazione dei dati. Questo significa che le modifiche apportate ai modelli di dati possono essere tracciate, esaminate e, se necessario, annullate, offrendo un livello di controllo e responsabilità.
3. Modularità: Promuove la modularità permettendo agli utenti di creare frammenti di codice SQL riutilizzabili chiamati "macro" e "modelli". Questi possono essere condivisi tra diversi progetti, garantendo coerenza ed efficienza nelle trasformazioni dei dati.
4. Test e Documentazione: DBT supporta il testing della qualità e della coerenza dei dati attraverso la creazione di test sui dati trasformati. Inoltre, genera automaticamente documentazione basata sul codice scritto, semplificando la comprensione e la collaborazione sui processi di trasformazione dei dati.
5. Integrazione: Viene fornita l'integrazione nativa con Dremio, utile per supportare le operazioni di trasformazione dei dati, direttamente dentro la sua piattaforma.

A livello architetturale, il ruolo di dbt all'interno dell'infrastruttura, e la sua integrazione con Dremio, può essere rappresentata come segue:



DBT all'interno della nostra infrastruttura

All'interno dell'architettura proposta, DBT verrebbe utilizzato per gestire la trasformazione dei dati caricati in Dremio. Questa tipologia di integrazione porterebbe molteplici funzionalità aggiuntive, che non sarebbero disponibili utilizzando esclusivamente Dremio all'interno dell'architettura, quali: il controllo delle versioni delle VDS, automatizzazione nella gestione di test e documentazioni e la gestione delle trasformazioni.

Inoltre, l'integrazione di DBT nella nostra architettura, permetterebbe di utilizzare Python per effettuare trasformazioni nei dati, oltre a SQL.

Collaborazione con Nessie

Project Nessie, abilita una modalità di interrogazione e salvataggio dei dati in maniera Git-like, attraverso la gestione dei metadati. In questo modo potremmo avere diverse versioni di una o più tabelle in base al branch di appartenenza. L'integrazione tra Nessie e dbt consentirebbe di materializzare i dati in base all'ambiente puntato da dbt e al branch di appartenenza di Nessie. Ad esempio, su dbt potremmo avere un branch di dev che punta ad un ambiente basso sul quale materializza i dati presenti nel branch di dev di Nessie, e un branch di prod di dbt che punta all'ambiente finale sul quale vengono materializzati i dati appartenenti al branch prod di Nessie. In questo modo possiamo promuovere in produzione solo i dati che hanno passato tutti i test preposti, ed eventualmente fare facilmente roll-back ad una versione precedente del dataset qualora si riscontrino degli errori, limitando al minimo i malfunzionamenti in produzione.

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.8 – Componente Spark

Apache Spark

Apache Spark è un framework di elaborazione distribuita, che gioca un ruolo cruciale nell'ecosistema di big data, fornendo una piattaforma per vari compiti di elaborazione dati, tra cui batch, iterazioni algoritmiche, query interattive e streaming. Ecco una panoramica dettagliata delle sue caratteristiche e della sua architettura:

Ruolo e Funzionalità:

- **Ruolo:** Spark è usato principalmente per l'elaborazione di grandi set di dati in modo veloce ed efficiente.
- **Funzionalità:** Offre supporto per il batch processing, l'elaborazione in tempo reale, l'apprendimento automatico, e l'analisi dei grafici. È noto per la sua velocità, grazie alla capacità di elaborare dati in memoria.

Caratteristiche Tecnologiche:

- **Elaborazione In-Memory:** Aumenta notevolmente la velocità di elaborazione delle applicazioni.
- **Tolleranza ai Guasti:** Utilizza i Resilient Distributed Datasets (RDD) per garantire la tolleranza ai guasti.
- **Scalabilità:** È progettato per essere scalabile su cluster con migliaia di nodi.
- **Linguaggi di Programmazione Supportati:** Offre API di alto livello in Java, Scala, Python e R.
- **Caching Potente:** Fornisce una semplice programmazione con potenti capacità di caching e persistenza su disco.

Struttura:

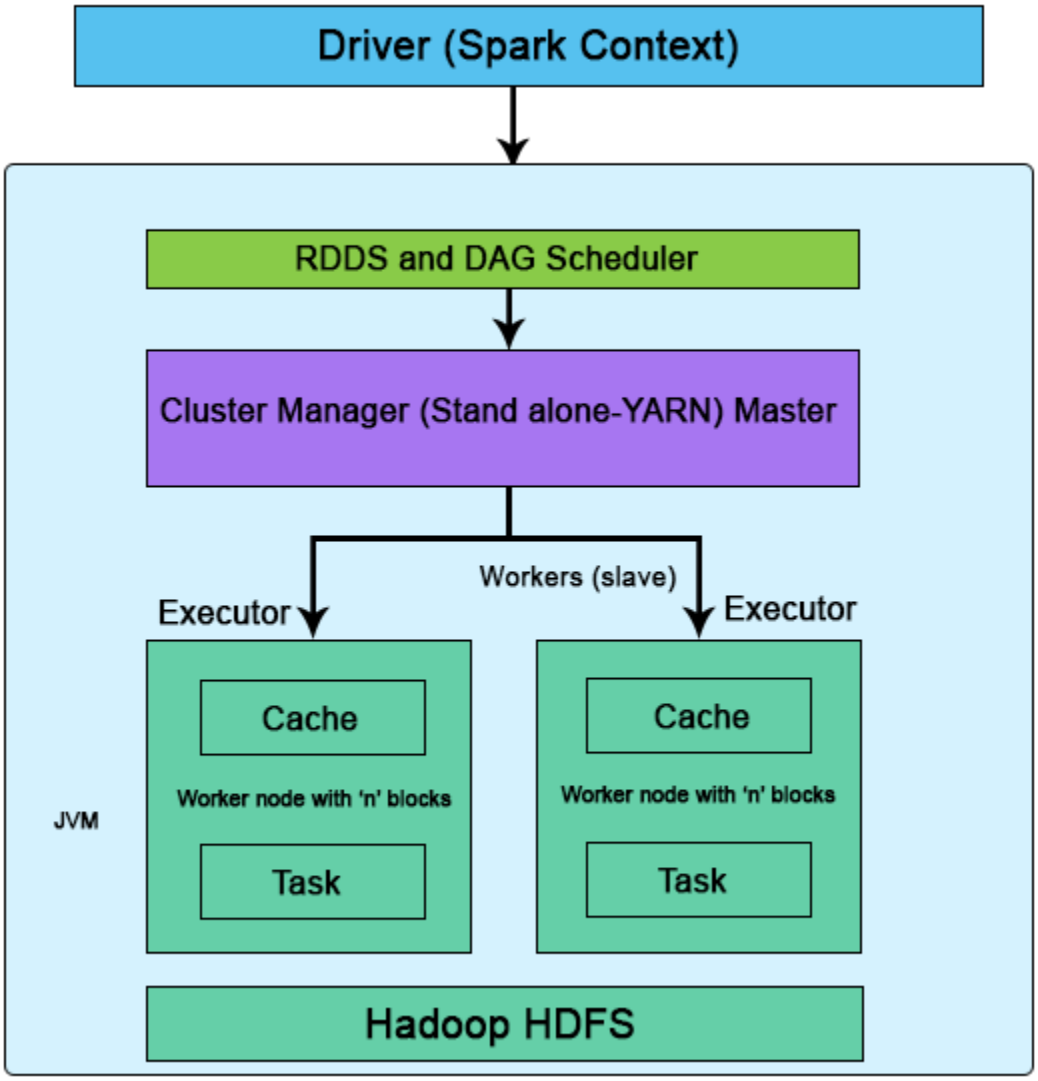
- **Architettura a Strati:** Composta da diversi componenti e strati, che sono liberamente accoppiati e integrati con varie estensioni e librerie.
- **Componenti Principali:** Spark Core, Spark SQL, Spark Streaming, MLlib e GraphX.

Sotto Componenti Principali:

- **Spark Core:** Base per l'elaborazione parallela e distribuita di grandi quantità di dati.

- **Spark SQL:** Integra l'elaborazione relazionale con la programmazione funzionale di Spark.
- **Spark Streaming:** Usato per l'elaborazione di flussi dati in tempo reale.
- **GraphX:** API di Spark per grafi e calcoli paralleli sui grafi.
- **MLlib (Machine Learning):** Libreria per machine learning in Apache Spark.

Architettura di Spark:



L'architettura di Spark è basata su due concetti fondamentali:

- **Resilient Distributed Dataset (RDD):** Un insieme di dati distribuiti e tolleranti ai guasti.
- **Directed Acyclic Graph (DAG):** Utilizzato per ottimizzare il flusso di lavoro.

Ruoli nel Cluster Spark:

- **Driver:** Gestisce il metadata su tutti i RDD e le loro partizioni, e converte le applicazioni degli utenti in unità esecutive più piccole chiamate task.
- **Executor:** Esegue i task e interagisce con i sistemi di archiviazione, fornendo anche storage in-memory per RDD.
- **Cluster Manager:** Responsabile per l'acquisizione delle risorse sul cluster Spark e la loro allocazione ai job di Spark. I tipi includono Standalone Cluster Manager, Hadoop YARN, Apache Mesos e Kubernetes.

Modi di Esecuzione:

- **Cluster Mode:** Il processo driver viene eseguito su un nodo worker all'interno del cluster.
- **Client Mode:** Il processo driver rimane sulla macchina client che ha inviato l'applicazione.
- **Local Mode:** Esegue l'intera applicazione Spark su una singola macchina

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.9 – Componente Jupyter e Zeppelin

Jupyter

Jupyter è costituito da una serie di prodotti open-source che permetteno di creare e condividere documenti che contengono codice live, simulazioni, visualizzazioni e testo narrativo. È ampiamente utilizzato per l'analisi di dati, il calcolo scientifico e l'apprendimento automatico. Ecco una descrizione dettagliata di Jupyter seguendo i punti da te indicati:

Ruolo e Funzionalità:

Ruolo: Jupyter è utilizzato principalmente per la data science, il machine learning, e l'insegnamento di materie scientifiche e tecniche. Fornisce un ambiente interattivo dove codice, output, visualizzazioni, e note possono essere mescolati.
Funzionalità: Supporta l'esecuzione di codice in diversi linguaggi di programmazione (Python, R, Julia, ecc.) e l'integrazione di dati, grafici, widget interattivi, e altro. È anche usato per la documentazione di progetti e per la condivisione di notebook interattivi attraverso vari formati.
Caratteristiche Tecnologiche:

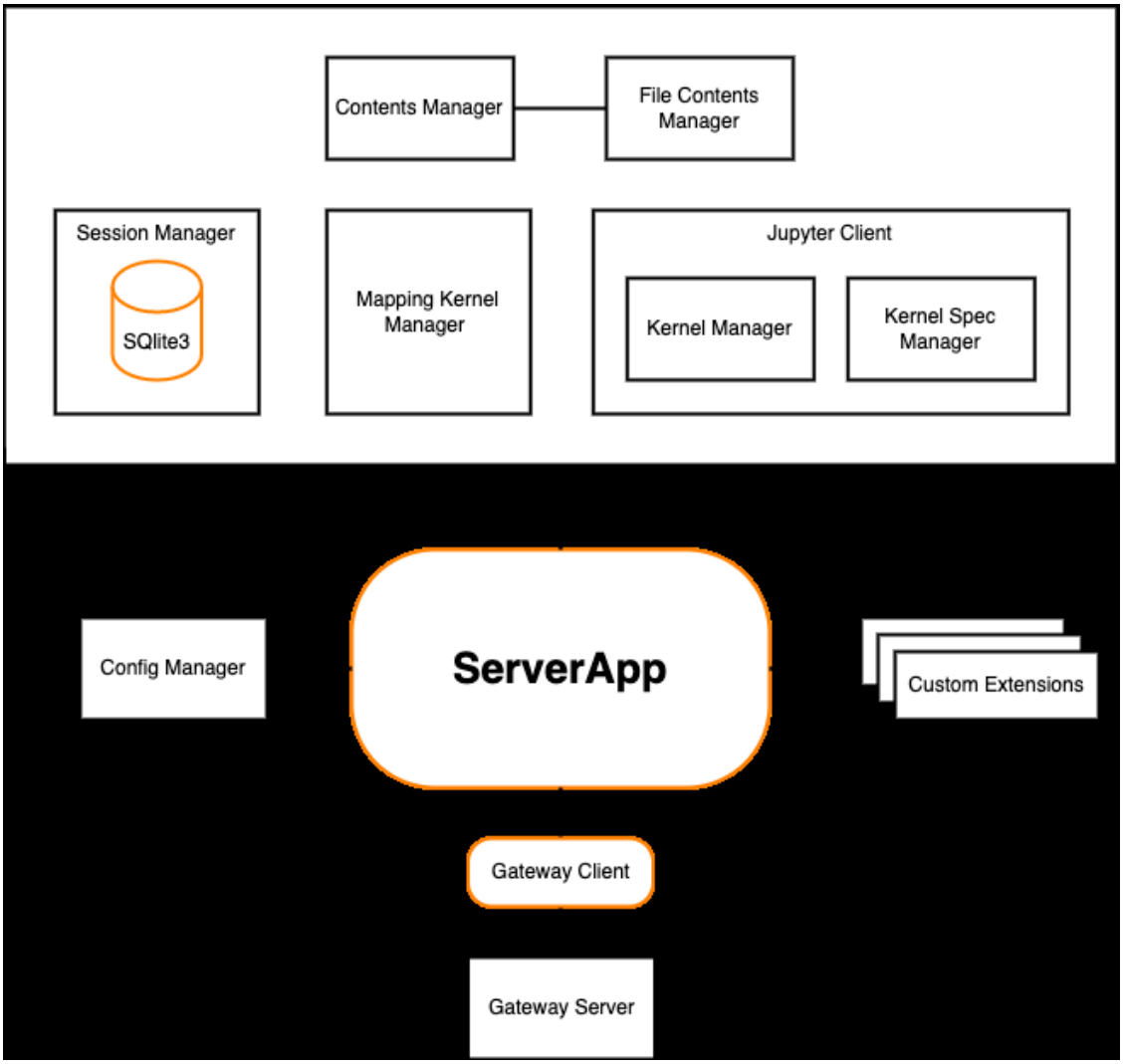
Interfaccia Web: Jupyter opera come un'applicazione web, rendendo facile l'accesso e la condivisione di notebook via browser.
Supporto Multipiattaforma: Funziona su diversi sistemi operativi (Windows, macOS, Linux).
Integrazione con Big Data: Può essere integrato con strumenti e piattaforme di big data, come Apache Spark.
Estensibilità: Supporta l'aggiunta di estensioni per funzionalità aggiuntive come il controllo versione, la formattazione del testo, ecc.
Struttura:

Kernel: Al cuore di ogni notebook Jupyter c'è un "kernel", un processo che esegue il codice scritto nel notebook. Ogni linguaggio di programmazione supportato da Jupyter ha il suo kernel specifico.
Interfaccia Utente: L'interfaccia utente è costituita da una serie di celle che possono contenere codice, testo (usando Markdown), equazioni (con LaTeX), o output grafici.
Sotto Componenti Principali:

Jupyter Notebook: Il componente principale, usato per creare e gestire i notebook.
JupyterHub: Una versione multiutente di Jupyter Notebook, che permette a più utenti di lavorare con i notebook su un server centralizzato.
JupyterLab: Un'interfaccia utente più avanzata che fornisce un ambiente di lavoro simile a un IDE per i notebook Jupyter, oltre ad altre forme di documenti e attività di sviluppo.

Fonti Utilizzate:

Segue il diagramma dell'architettura:



Architettura
L'architettura di Jupyter si basa su una serie di componenti interconnessi che facilitano l'esecuzione e la gestione di codice e contenuti in un ambiente di notebook interattivo. Ecco una panoramica dell'architettura di Jupyter, con alcuni diagrammi per aiutare a visualizzarla:

Architettura di Base di Jupyter
Kernel IPython: Il kernel IPython (IPykernel) fornisce calcolo e comunicazione con le interfacce frontend, come il notebook. Questo kernel è un processo separato responsabile per

l'esecuzione del codice utente e per cose come il calcolo di completamenti possibili. I frontend, come il notebook o la console Qt, comunicano con il kernel IPython utilizzando messaggi JSON inviati tramite socket ZeroMQ. Il kernel condivide il meccanismo di esecuzione centrale con l'IPython terminale.

Jupyter Notebook Interface: L'interfaccia del Jupyter Notebook estende il notebook oltre il codice alla visualizzazione, multimedia, collaborazione e altro. Quando si salva un notebook, questo viene inviato dal browser al server Jupyter, che lo salva su disco come un file JSON con estensione .ipynb. Il server Jupyter agisce come un hub di comunicazione, collegando il browser, il file del notebook su disco e il kernel.

Architettura del Server Jupyter
ServerApp: Applicazione principale basata su Tornado che collega tutti i componenti insieme.
Extensions e Gateway Server: Permettono la creazione di endpoint REST API personalizzati e forniscono l'accesso ai kernel Jupyter eseguiti su altri host.
Contents Manager e File Contents Manager: Responsabili per la gestione dei notebook sul file system.
Session Manager: Gestisce le sessioni degli utenti, avviando un processo per fornire il kernel per l'utente e generando un nuovo ID sessione per ogni notebook aperto.

Zeppelin

Apache Zeppelin è un'applicazione web open source per la visualizzazione e la collaborazione interattiva per l'analisi dei dati. È particolarmente utilizzata in ambienti di data science e big data per lavorare con set di dati e visualizzazioni in tempo reale. Ecco una descrizione dettagliata di Apache Zeppelin seguendo i punti da te indicati:

Ruolo e Funzionalità:

Ruolo: Zeppelin funge da ambiente di notebook per l'analisi dei dati, consentendo agli utenti di scrivere, eseguire e condividere codice in vari linguaggi di programmazione.

Funzionalità: Supporta diversi interpreti, come Apache Spark, Python, JDBC, Markdown e Shell. Gli utenti possono eseguire codice in questi linguaggi, visualizzare risultati sotto forma di grafici e tabelle interattive e collaborare con altri utilizzando funzionalità di condivisione.
Caratteristiche Tecnologiche:

Multi-linguaggio: Zeppelin consente di usare diversi linguaggi di programmazione e interpreti

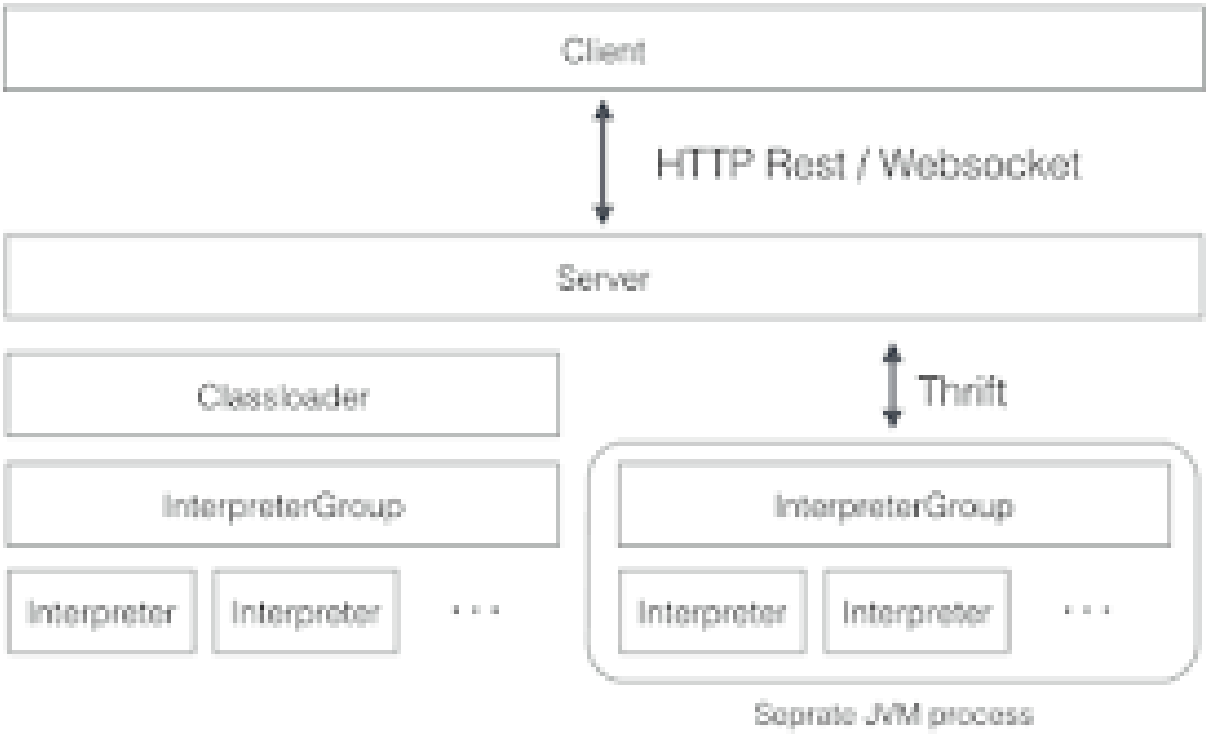
nello stesso notebook.
Integrazione Big Data: Si integra facilmente con piattaforme di big data come Apache Spark, Hadoop, etc.

Visualizzazioni Interattive: Fornisce una vasta gamma di grafici e visualizzazioni interattive.
Collaborazione: Supporta la collaborazione in tempo reale, consentendo a più utenti di lavorare sullo stesso notebook.
Struttura:

Notebook: I notebook sono i documenti principali in Zeppelin e possono contenere codice, testo, visualizzazioni e risultati.
Interpreti: Zeppelin utilizza interpreti per eseguire codice in vari linguaggi. Ogni interprete funziona come un plug-in e può essere configurato a livello di notebook o utente.

Zeppelin può essere esteso attraverso l'aggiunta di interpreti specifici per il linguaggio di riferimento.

Segue il diagramma dell'architettura della componente Zeppelin



Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.10 – Componente Data Catalog

Data Catalog

Il Data Catalog si occupa di raccogliere tutti i metadati della Data Platform in un unico posto. Può essere considerata una repository centralizzata e unificata che ha il ruolo di unica fonte di verità riguardo i dati dell'organizzazione. Permette di raccogliere i metadati provenienti da diversi fonti come il Data Lake, Database Relazionali, pipeline orchestrator e strumenti di visualizzazione. Le caratteristiche principali di un Data Catalog sono:

- Metadata Repository;
- Data Discovery;
- Data Governance;
- Indicatori di Data Quality;
- Strumenti di collaborazione;
- Data Lineage;
- Data History;
- Controlli di sicurezza;
- Integrazioni con data source e tool.

Per queste ragioni diventa fondamentale l'utilizzo di un Data Catalog quando i dati gestiti da una organizzazione sono complessi e provengono da più fonti, così da avere in un unico posto l'accesso a tutte queste informazioni. Al momento in cui viene scritto il presente documento, i due tool Open Source più validi per questo ruolo sono:

- DataHub
- Open Metadata

Abbiamo deciso di includere entrambi gli strumenti in questo documento in quanto molto simili tra loro, a parte alcuni dettagli che li contraddistinguono. Inoltre, per certi strumenti è importante valutare anche la user experience di uno strumento rispetto che l'altro, e può essere valutata solo dopo un periodo consistente di test.

DataHub

DataHub è un Data Catalog open-source che offre tutte le feature principali indicate nel paragrafo precedente. Tramite interfaccia grafica, quindi, è possibile:

- Effettuare operazioni di Search e Discovery;

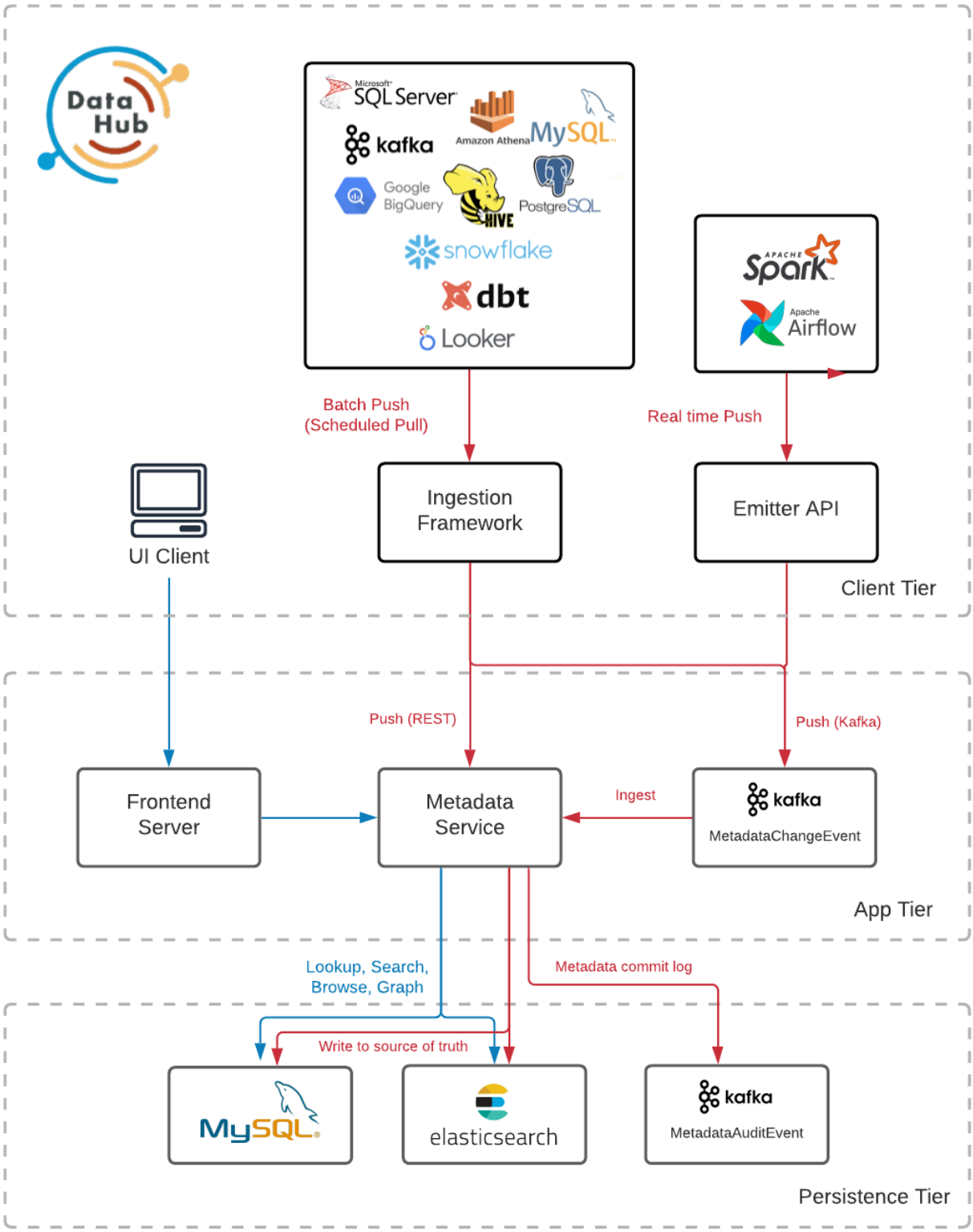
- Effettuare Data Lineage end-to-end;
- Avere le integrazioni con molte sorgenti e tool come ad esempio Airflow, dbt, Elasticsearch, Hive, Iceberg, Kafka, LDAP, NiFi, OpenAPI, Okta, Oracle, MySQL, MS-Server, MongoDB, S3, Spark, Superset etc. La lista completa può essere visualizzata qui.
- Effettuare Data Governance in tempo reale grazie all'abilitazione di notifiche, all'integrazione con altri sistemi come Jira o avere log di audit per conoscere le azioni effettuate dagli utenti. Inoltre, è possibile specificare il proprietario di una determinata entità, oltre che gestire le entità con i Tag o con un glossario condiviso;
- Effettuare operazioni di amministrazione gestendo gli utenti attraverso Policy e Ruoli, per definire la corretta visibilità dei metadati;
- Effettuare l'ingest dei metadati da interfaccia grafica;
- Schema History per conoscere i cambiamenti avvenuti ad un determinato dataset.

Architettura

DataHub è composto dalle seguenti componenti:

- Metadata Store: vengono utilizzati MySQL oppure Postgres, Elasticsearch e Kafka per salvare i dati ed indicizzarli;
- Metadata Model: viene utilizzato lo Schema Registry di Kafka per gestire il cambiamento dei metadati;
- Ingestion Framework: viene utilizzato Kafka per effettuare l'ingestion dei metadati;
- GraphQL API: vengono esposte le API di GraphQL per navigare in maniera più agevole tra i metadati;
- User Interface: viene esposta una interfaccia UI da cui possiamo interagire con il Data Catalog.

L'architettura è schematizzata nella seguente figura:


Data Hub
Microsoft SQL Server
kafka
Amazon Athena
MySQL
Google BigQuery
HIVE
PostgreSQL
snowflake
dbt
Looker
Apache Spark
Apache Airflow
Batch Push
(Scheduled Pull)
Real time Push
UI Client
Ingestion Framework
Emitter API
Client Tier
Push (REST)
Push (Kafka)
Frontend Server
Metadata Service
Ingest
kafka
MetadataChangeEvent
App Tier
Lookup, Search, Browse, Graph
Metadata commit log
Write to source of truth
MySQL
elasticsearch
kafka
MetadataAuditEvent
Persistence Tier

Metadata Ingestion

DataHub permette l'ingestion dei metadati nelle seguenti forme:

- Push-based;
- Pull-based;
- Tramite API.

La tecnica push-based permette di inviare i metadati direttamente dalle sorgenti verso DataHub appena vi è un cambiamento degli stessi, mentre la modalità pull-based permette di estrarre i metadati dalle sorgenti in maniera incrementale o in batch. Infine, è possibile effettuare il push dei metadati tramite REST API.

Modalità di deployment

DataHub è disponibile nelle sole versioni containerizzate. Quindi, oltre che su Kubernetes (sia on-prem che managed) è possibile installarlo con docker compose.

Autenticazione

DataHub offre diversi metodi di autenticazione per gli utenti. Essa può avvenire:

- Quando l'utente vuole effettuare il login tramite interfaccia;
- Quando l'utente effettua delle chiamate API al backend.

Per quanto riguarda l'autenticazione al frontend, sono supportati i seguenti metodi:

- Autenticazione tramite db: si creano gli utenti tramite interfaccia di DataHub che ricevono un invito. Le informazioni sono salvate all'interno del db di DataHub.
- Single Sign-On con OpenID Connect: l'autenticazione avviene tramite strumenti di terze parti come Okta, Keycloak oppure Google/Azure Authenticator. Questa è la modalità preferita in ambienti di produzione.
- Autenticazione JaaS: permette di utilizzare un protocollo di autenticazione custom.

Per quanto riguarda l'autenticazione per le chiamate API, l'utente deve utilizzare dei token che possono essere di due tipi:

- Session Token: in genere durano 24 ore e contengono informazioni circa i cookie utilizzati in una certa sessione.

- Personal Access Token: vengono generati dall'interfaccia di DataHub ed in genere hanno durata più lunga rispetto ai Session Token, che li rende ideali per essere utilizzati dagli applicativi.

Open Metadata

Open Metadata è un Data Catalog open-source. Le principali feature presenti in Open Metadata sono:

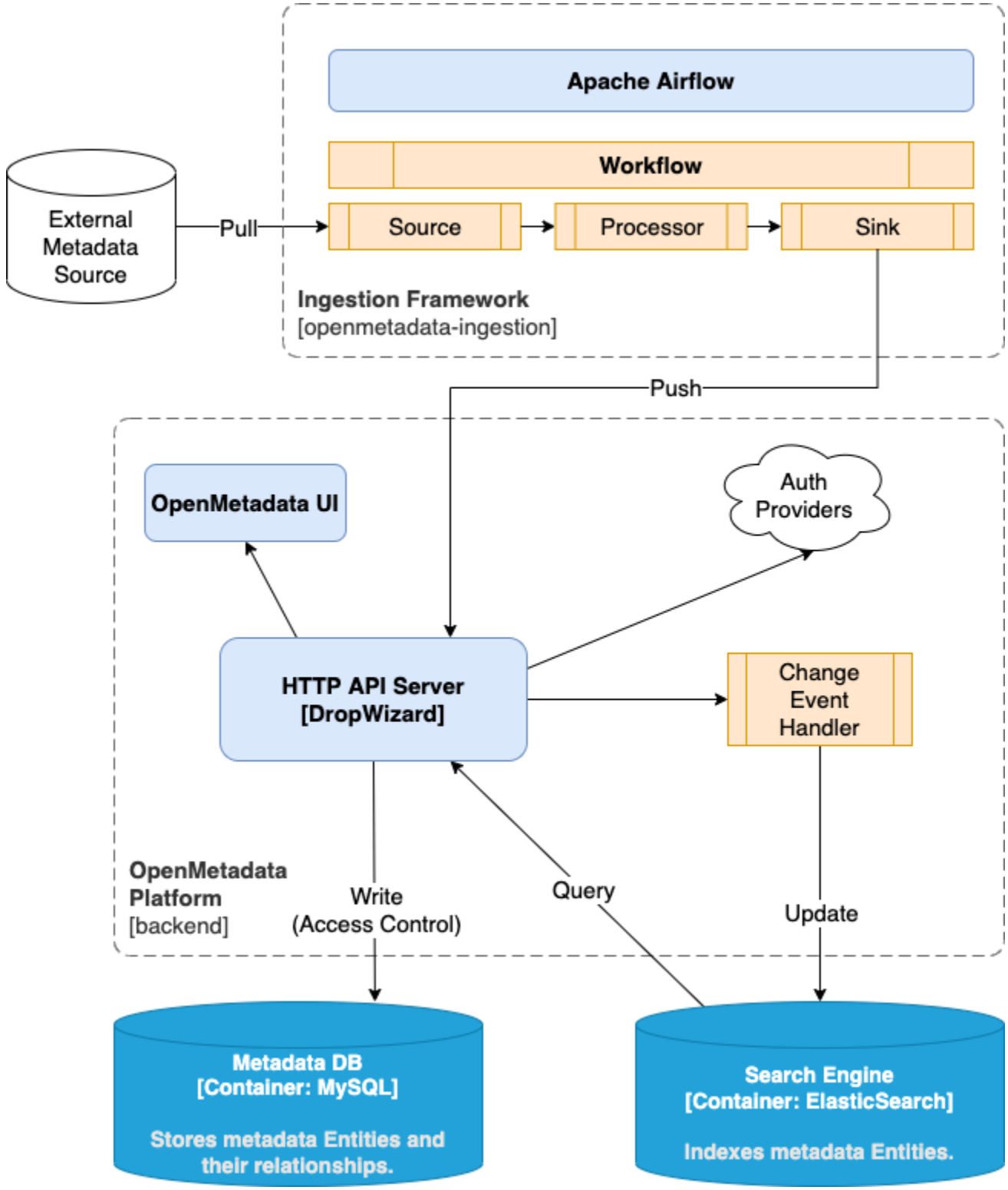
- Data discovery e data search da un unico strumento;
- Activity Feed per tracciare i cambiamenti ai dati che ci interessano (di cui siamo proprietari o su cui abbiamo privilegi);
- Notifiche via Webhook dei cambiamenti avvenuti ai dati;
- Permette di effettuare una soft delete delle tabelle, in modo da visualizzarle come eliminate;
- RBAC policy per gestire gli utenti in base al ruolo che ricoprono nell'organizzazione;
- Data Lineage con integrazione con dbt che permette di vedere quale modulo è stato usato per generare la tabella;
- Metadata Versioning.

Architettura

L'architettura di Open Metadata è la seguente:

- JsonSchemas per i cambiamenti dello Schema dei metadati;
- Dropwizard e Jetty per le REST API;
- MySQL oppure Postgres per salvare i metadati;
- Elasticsearch per indicizzare i metadati e reperirli.

Nell'immagine seguente vi è una schematizzazione della stessa:


Apache Airflow
Workflow
External Metadata Source
Pull
Source
Processor
Sink
Ingestion Framework
[openmetadata-ingestion]
Push
OpenMetadata UI
Auth Providers
HTTP API Server
[DropWizard]
Change Event Handler
OpenMetadata Platform
[backend]
Write
(Access Control)
Query
Update
Metadata DB
[Container: MySQL]
Stores metadata Entities and their relationships.
Search Engine
[Container: ElasticSearch]
Indexes metadata Entities.

Metadata Ingestion

La modalità di ingestion possibile su Open Metadata è pull-based.

Modalità di deployment

Open Metadata può essere installato su macchine bare metal oppure come servizio containerizzato sia con docker compose che su Kubernetes (on-prem o managed).

Autenticazione

Open Metadata supporta diversi metodi di autenticazione:

- Basic Authentication: l'accesso viene effettuato tramite username e password e le credenziali vengono salvate nel db di Open Metadata.
- LDAP: è possibile effettuare l'accesso su Open Metadata utilizzando le credenziali LDAP.
- Single Sign-On: è possibile accedere anche collegando ad Open Metadata un sistema esterno che gestisca l'autenticazione come Auth0, Custom OIDC, Keycloak oppure Azure/Google Authenticator.
- SAML: è possibile configurare l'autenticazione SAML con quasi ogni IDP presente.

Se è stata attivata l'autenticazione SSO, è possibile staccare un token JWT ed utilizzare questo per accedere alle API di Open Metadata

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.11 – Componente Opensearch

Monitoraggio

Al fine di monitorare lo stato delle varie macchine che verranno utilizzate ed i servizi che verranno installati all'interno dell'infrastruttura, verrà effettuato il deployment di un database di monitoraggio.

Essendo i dati di monitoraggio strutturati come log; disomogenei e provenienti da diverse fonti, l'utilizzo di un database consentirebbe di organizzare tali dati in modo coerente, rendendoli più facili da capire e da utilizzare.

In secondo luogo, tale base dati verrà utilizzata per effettuare interrogazioni, elaborazioni, reportistica e, se necessario, applicare algoritmi di Machine Learning ai dati di monitoraggio. Tutto questo, al fine di identificare tendenze, anomalie, o altri pattern.

Opensearch

OpenSearch è un motore di ricerca e analisi full text, basato su licenza open source. La sua architettura di basa su un paradigma distribuito, ovvero facilmente scalabile. Il suo protocollo di comunicazione è basato su REST. È un fork della versione 7 di Elasticsearch.

Le principali caratteristiche di OpenSearch includono:

- Scalabilità: OpenSearch è progettato per essere scalato in modo elastico, in base alle esigenze di dati e carico.
- Velocità: OpenSearch è in grado di eseguire ricerche su grandi quantità di dati in modo rapido e efficiente.
- Flessibilità: OpenSearch supporta una vasta gamma di formati di dati e tipi di analisi.

OpenSearch può essere utilizzato per una varietà di scopi, tra cui:

- Ricerche full text: OpenSearch può essere utilizzato per cercare testo in documenti, log e altri dati.
- Analisi dei dati: OpenSearch può essere utilizzato per analizzare dati per identificare tendenze e pattern.
- Monitoraggio: OpenSearch può essere utilizzato per monitorare lo stato di sistemi e applicazioni.

Metricbeat

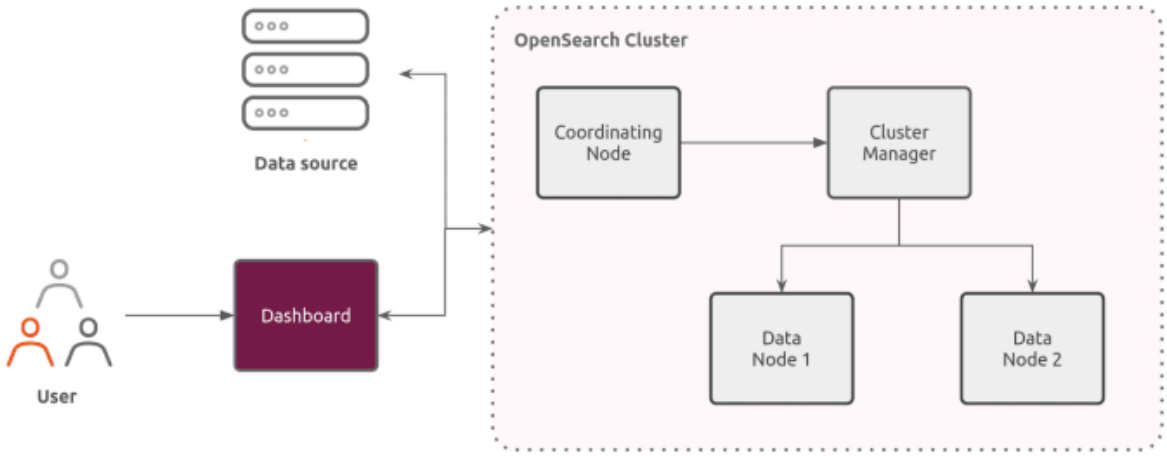
Al fine di salvare metriche di monitoraggio in Opensearch, verrà utilizzato Metricbeat come data shipper, il quale leggerà i dati di log dei vari applicativi, e li invierà al cluster Opensearch. Metricbeat è un servizizo basato su paradigma basato sugli agenti in push.

Opensearch tiering

Opensearch, nella sua versione on-prem offre tutte le funzionalità in maniera completamente gratuita. In Elasticsearch alcune funzionalità (come l'advanced security, i moduli Machine Learning, le searchable snapshots, ...) possono essere utilizzate esclusivamente con le versioni a pagamento, mentre in OpenSearch l'utilizzo di tutte queste funzionalità è gratuito, anche nelle distribuzioni on-prem.

Architettura

Segue lo schema architetturale di Opensearch:



Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.12 – Componenti Cartografiche

Vengono di seguito descritte le componenti cartografiche della data platform.

GDAL

GDAL (Geospatial Data Abstraction Library) è una libreria open source di conversione per dati geografici raster e vettoriali. È pubblicata dalla Open Source Geospatial Foundation (OSGeo) con licenza X/MIT.

Questa libreria presenta un singolo modello di dati astratto sia per dati raster che vettoriali, fornendo un'interfaccia uniforme alle applicazioni che la utilizzano.

E' possibile utilizzarla sia tramite riga di comando mediante opportuni eseguibili (ogr2ogr, gdal_translate, ecc...) che programmaticamente attraverso una serie di binding per diversi linguaggi tra cui Python.

E' una delle librerie GIS più utilizzate sia in ambito open-source che proprietario e gestisce tutti i principali formati raster e vettoriali esistenti.

La velocità di implementazione di nuovi formati è molto elevata e permette alle applicazioni di utilizzarli in tempi rapidi.

In particolare il supporto dei formati di tipo cloud-native, come il formato COG (Cloud Optimized Geotiff) e il formato GeoParquet, sono particolarmente indicati per l'utilizzo all'interno della data platform.

Nell'ambito della data platform GDAL si occupa della conversione del formato per i dati geografici nella fase di ingestion dei dati e in quella di write-back sui sistemi esterni, ad esempio per rendere disponibili ai servizi della Infrastruttura di Dati Territoriale i dati elaborati dalla piattaforma.

DuckDB

DuckDB è un database relazionale in-process, orientato alle elaborazione analitica dei dati.

E' distribuito con licenza MIT.

Alcuni punti chiave:

1. Semplice e portabile: DuckDB non ha dipendenze esterne. È un sistema serverless che può essere incorporato direttamente nelle applicazioni.
2. Ampio supporto SQL: DuckDB offre un ricco dialetto SQL, con funzionalità che vanno oltre il semplice SQL. Supporta subquery nidificate, window functions e tipi complessi come array e strutture.
3. Ottimizzato per l'analisi: DuckDB è progettato per l'elaborazione analitica. Utilizza un motore vettorializzato e parallelo, consentendo l'elaborazione di grandi moli di dati.
4. Sorgenti dati supportate: È uno strumento utile per l'analisi dati e l'interrogazione di dataset tabulari da file in vari formati, tra cui Parquet, CSV e JSON, utilizzabili in modalità cloud-native mediante SQL.
5. Bindings: DuckDB ha bindings diversi linguaggi tra cui Python, R, Java, Julia, ecc.

Sebbene DuckDB non sia specificamente un sistema di analisi geospaziale, presenta una "Spatial Extension" che implementa uno specifico tipo "GEOMETRY" basato sul modello standard "OGC/ISO Simple Features".

L'estensione fornisce inoltre un ampio set di funzioni geografiche simili a quelle presenti in PostGIS. Inoltre permette, attraverso GDAL, la lettura e la scrittura di oltre 50 formati vettoriali e la loro analisi in maniera trasparente, come se fossero tavole di database interne.

Nell'ambito della data platform DuckDB puà essere utilizzato per analisi di tipo geografico e/o generico, integrabili nelle pipeline di elaborazione e nell'ambiente di data exploration.

L'integrazione con GDAL ne permette l'utilizzo anche in fase di ingestion o write-back.

Geopandas

Geopandas è un progetto open source che estende Pandas per poter lavorare con dati geografici in Python, estendendone i tipi per consentire operazioni spaziali su tipi geometrici.

GeoPandas supporta una vasta gamma di operazioni geometriche: buffering, intersezione, unione calcoli di arre e lunghezze semplificazione, trasformazione e validazione.

Permette di effettuare in ambiente Python le stesse analisi effettuabili su un sistema GIS ed è molto utilizzato in ambienti di tipo notebook.

Nell'ambito della data platform Geopandas può essere utilizzato per analisi spaziali nell'ambiente di esplorazione.

Leafmap

Leafmap è un pacchetto Python open source disegnato per rendere facilmente accessibili le analisi geospaziali interattive e la visualizzazione cartografica in ambiente Jupyter, utilizzabile anche da utenti con competenze di programmazione limitate.

Leafmap offre diverse funzionalità di analisi geospaziale, alcune delle principali funzionalità supportate sono :

1. Visualizzazione di dati geospaziali: Leafmap consente di visualizzare dati vettoriali e raster su mappe interattive senza scrivere codice. Permette di creare visualizzazioni tematiche dei dati e di interrogare in maniera interattiva i dati visualizzati.
2. Analisi di dati vettoriali: si possono eseguire operazioni di analisi su dati vettoriali (calcolo aree, lunghezze, intersezioni, buffer e altre operazioni geometriche). All'interno di Leafmap è integrato e utilizzabile direttamente nell'interfaccia il backend analitico WhiteboxTools, una potente piattaforma di analisi con oltre 480 strumenti di processamento disponibili.
3. Analisi di dati raster: Leafmap supporta l'analisi di dati raster, inclusi calcoli di statistiche, estrazione di valori pixel, maschere e altre operazioni basate su raster.

Leafmap supporta vari formati di dati vettoriali come Shapefile, GeoJSON, GeoParquet e qualsiasi formato supportato da GeoPandas.

Nell'ambito della data platform Leafmap si occupa dell'analisi spaziale e della visualizzazione cartografica nell'ambiente di data exploration.

deck.gl

deck.gl è un framework javascript basato su **WebGL2** per l'analisi esplorativa e la visualizzazione di dataset geografici.

E' un client cartografico molto performante nell'effettuare il rendering e l'aggiornamento di grandi dataset, grazie al supporto di calcoli ad alte prestazioni sulla GPU.

Inoltre le API di deck.gl utilizzano un paradigma di programmazione di tipo reactive, consentendo un rendering efficiente anche con carichi di dati pesanti e l'integrazione in ambiente React.

Il supporto di livelli di tipo WMS ne permette l'integrazione con la IDT regionale, mentre il supporto a livelli ti tipo TileLayer ne permette l'integrazione con servizi di mappe di base.

E' utilizzabile anche in ambiente Jupyter mediante il pacchetto pydeck che offre un binding python alle API di deck.gl sia direttamente che attraverso leafmap.

Nell'ambito della data platform deck.gl può essere utilizzato per realizzare front-end cartografici orientati alla visualizzazione di grandi moli di dati sia in come client dedicati che integrati nell'ambiente di esplorazione notebook.

Essendo uno strumento orientato alla data analytics offre strumenti di visualizzazione migliori dei client cartografici generici come Leaflet e OpenLayers e complementari a questi, estendo quindi le funzionalità generali della infrastruttura di dati territoriali regionali.

PBI-Carto

Si tratta si un componente custom sviluppato internamente che permette di inserire in report PowerBI visualizzazioni cartografiche tematiche dei dati ed è dotato di una integrazione con l'infrastruttura cartografica regionale.

In particolare permette di:

- Visualizzare dati in formato GeoJSON permettendone la rappresentazione tematica in base ai valori di un campo del dataset.
- Rappresentare su mappa e tematizzare dataset contenenti le coordinate (longitudine e latitudine).
- Visualizzare una mappa di sfondo a scelta tra un ampio set di stili.
- Visualizzare livelli cartografici del sistema cartografico regionale mediante i relativi servizi WMS.

Nell'ambito della data platform questo componente permette di estendere e personalizzare le funzionalità cartografiche di PowerBI rispondendo meglio ai requisiti degli utenti.

Liguria Digitale

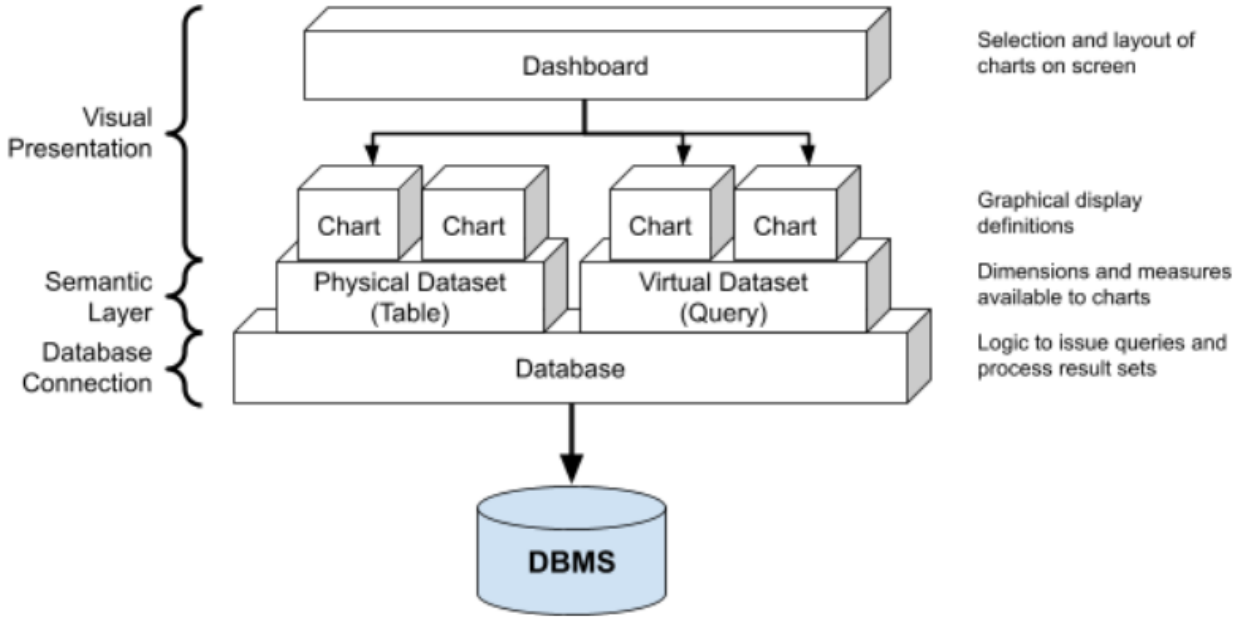
Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.13 – Componente Superset

Apache Superset

Apache Superset è un'applicazione web open source di business intelligence (BI), che consente agli utenti di esplorare, visualizzare ed interrogare i dati in modo efficiente e sicuro. Può essere utilizzato da Data Analyst, Data Scientist e utenti aziendali di tutti i livelli di abilità.

A livello architetturale, il funzionamento di Apache Superset può essere rappresentato come segue:



Tra le sue caratteristiche principali abbiamo:

- Connessioni dati versatili: Superset si integra con una vasta gamma di sorgenti dati, tra cui database SQL (come MySQL, PostgreSQL, etc.), data warehouse (come Snowflake, BigQuery), file CSV e molti altri.
- Interfaccia utente intuitiva: La sua interfaccia utente è molto intuitiva e permette agli utenti di creare facilmente visualizzazioni interattive come grafici, mappe, tabelle pivot e dashboard senza scrivere codice.

- Creazione di dashboard dinamiche e interattive: Gli utenti possono creare dashboard personalizzate combinando diverse visualizzazioni e controlli interattivi come filtri, dropdown e selezioni temporali per esplorare i dati in profondità.
- Sicurezza e controllo degli accessi: Superset offre funzionalità avanzate di sicurezza e controllo degli accessi per gestire i permessi degli utenti, garantendo che solo le persone autorizzate possano accedere e manipolare i dati.
- Controllo all'accesso delle righe dei dataset: Tramite Superset, è possibile definire un insieme di regole che gestiscano la modalità di accesso a determinate porzioni (righe) di un dataset.
- Superset, è inoltre dotato di una community molto attiva e collaborativa, in grado di fornire supporto agli analisti e sviluppatori.

UI

L'interfaccia di Superset è implementata in TypeScript; per lo sviluppo dei componenti viene utilizzato React e Redux per gestire l'app/global state.

Sicurezza e Autenticazione

Superset supporta l'integrazione con i principali backend di autenticazione: database, OpenID, LDAP, OAuth, REMOTE_USER, etc.

Integrazione con Dremio

Tramite il connettore sqlalchemy_dremio, è possibile connettere un'istanza di Dremio a Superset. Tramite questo connettore è possibile effettuare la connessione a Dremio in due modalità:

- Tramite il protocollo ODBC, alla porta 31010
- Tramite il protocollo Arrow Flight, alla porta 32010

Dashboard embedding

Utilizzando Superset, è possibile effettuare l'embedding delle dashboard all'interno di applicazioni React. Nel dettaglio, l'embedding viene effettuato tramite l'inserimento di un iframe, contenente una pagina Superset in un'applicazione host.

Liguria Digitale

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

5.14 – Componente Opendata Downloader E Api end-point

Componente downloader

La componente "downloader" è stata concepita per far fronte alla necessità di integrare gli strumenti offerti dalla data platfom con il nuovo sistema open data, al fine di rendere i dati disponibili per il download.

La funzionalità principale è quella di permettere all'utente di scaricare i dati presenti nel Data Lake che sono stati curati e validati per la pubblicazione.

Caratteristiche tecnologiche:

In quanto questa componente dovrà essere sviluppata durante lo svolgimento del progetto, sarà necessario definire il perimetro e le caratteristiche in maniera puntuale successivamente ad una fase di analisi dettagliata, dipendente dalla scelta degli strumenti infrastrutturali effettuata a livello aziendale.

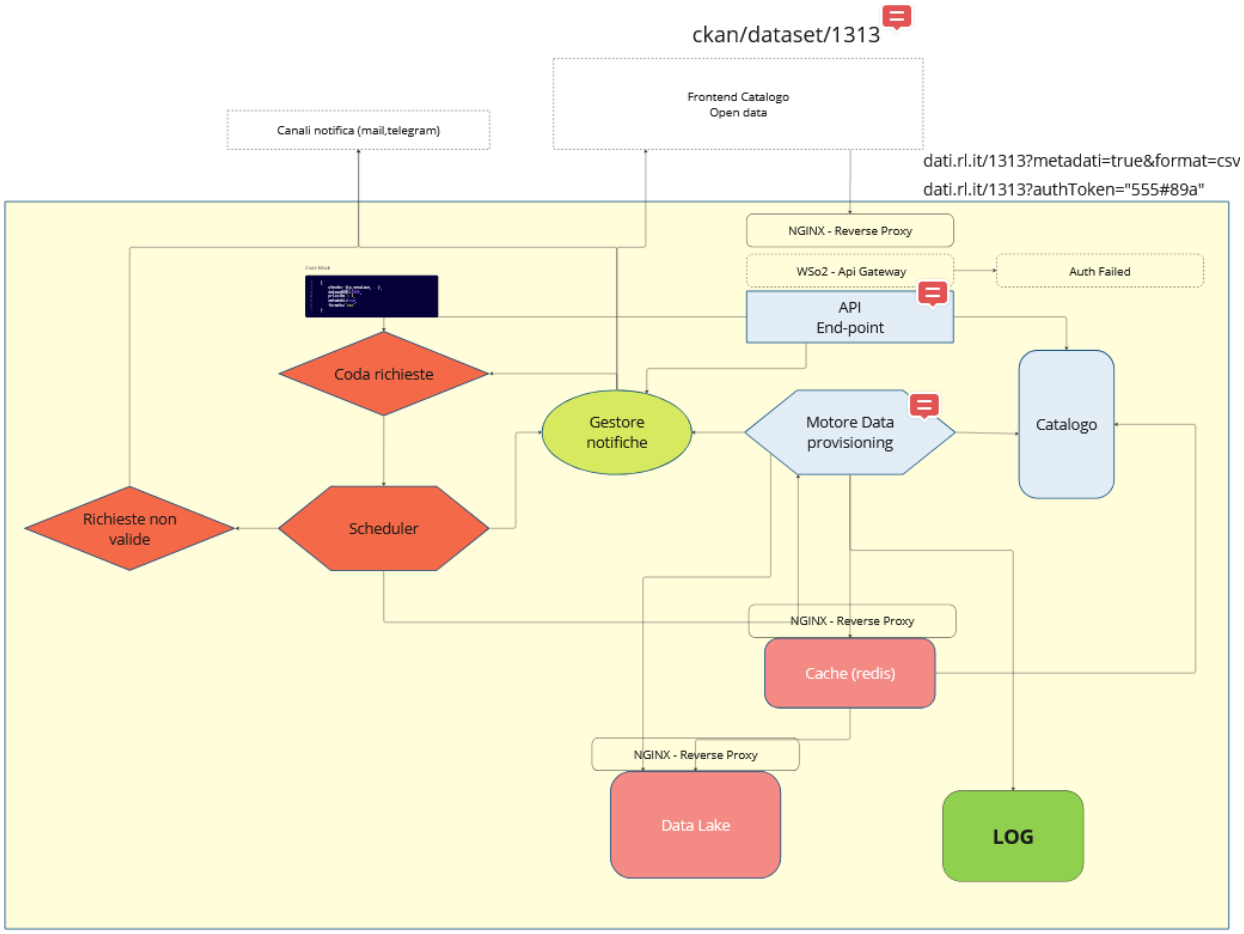
Al momento si intravedono due potenziali approcci alla definizione di un'architettura.

Il primo prevede la gestione delle richieste di scarico attraverso delle code di messaggi gestite a livello applicativo e di servizi generici che possano portare a termine le richieste degli utenti.

Nel secondo caso, la gestione delle richieste avviene a livello infrastrutturale mediante l'utilizzo di un sistema di provisioning automatico. Questo sistema crea microservizi temporanei e specializzati per ciascuna richiesta in modo da gestire le richieste in modo efficiente e personalizzato.

La differenza deve essere un tradeoff tra l'impatto infrastrutturale e di gestione con i tempi di sviluppo delle componenti applicative e il mantenimento.

A riprova della complessità dell'operazione viene presentato un diagramma preliminare della soluzione applicativa:


ckan/dataset/1313
Frontend Catalogo
Open data
Canali notifica (mail,telegram)
dati.rl.it/1313?metadati=true&format=csv
dati.rl.it/1313?authToken="555#89a"
NGINX - Reverse Proxy
WSo2 - Api Gateway
Auth Failed
API
End-point
Coda richieste
Gestore
notifiche
Motore Data
provisioning
Catalogo
Richieste non
valide
Scheduler
NGINX - Reverse Proxy
Cache (redis)
NGINX - Reverse Proxy
Data Lake
LOG

Sede Legale:
VIA MELEN 77 - 16152 GENOVA
Tel +39 - 01065451 - Fax +39 - 0106545422

Componente Api REST

La data platform ha come obiettivo quello di rendere disponibili potenzialmente tutti i dati contenuti al suo interno a sistemi esterni come ad esempio il PDND. Questa componente si occupa di offrire servizi REST di accesso ai dati per garantire l'interoperabilita' della piattaforma con sistemi gia' in produzione e futuri sviluppi senza la necessita' di sviluppare ogni volta un servizio specifico.

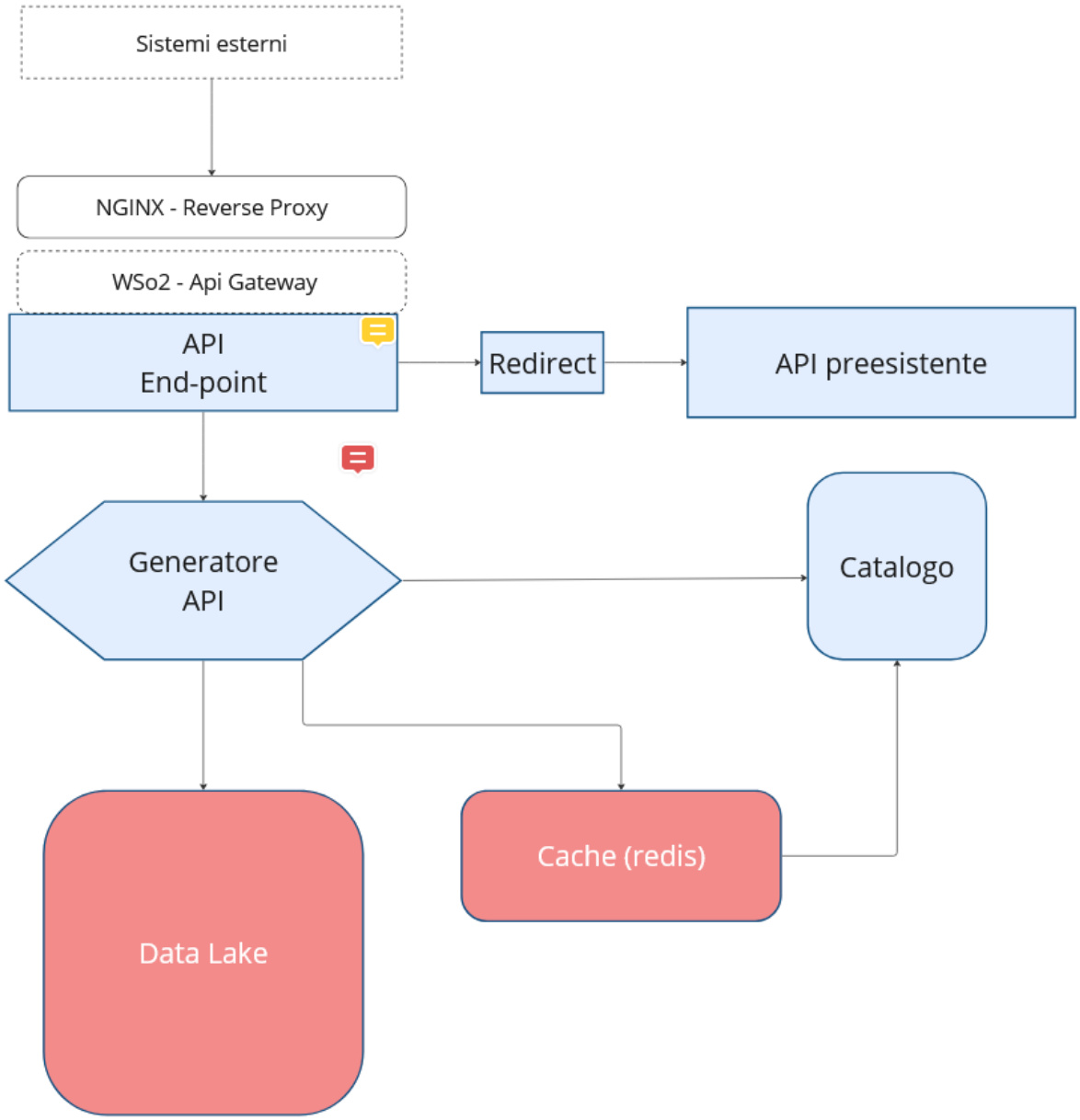
Carattteristiche tecnologiche

Il primo passo dello sviluppo prevede l'integrazione con l'api gateway aziendale basato su WSO2 che copre anche la funzionalita' di Identity manager cosi' da permettere la corretta autenticazione e autorizzazione delle richieste fatte, al fine di proteggere la riservatezza delle informazioni contenute e di controllare gli accessi.

In seguito sara' necessario sviluppare un framework per la generazione e configurazione dinamica degli end-point di accesso ad ogni dataset, e le componenti necessarie alla gestione dei servizi.

Per garantire un'integrazione completa, queste componenti dovranno essere interconnesse con le interfacce dei cataloghi esistenti. In questo modo, sarà possibile accedere alle informazioni e arricchire con i metadati relativi ai servizi offerti.

Segue un diagramma preliminare dell’architettura della componente.



Dopo l'analisi iniziale, sarà necessario prendere decisioni riguardanti l'integrazione delle API che già forniscono servizi all'interno di questa architettura. Inoltre, sarà di fondamentale importanza considerare le scelte infrastrutturali che porteranno alla realizzazione di dettaglio, tenendo conto dei trade-off individuati nella fase precedente.